

INDIRA GANDHI NATIONAL OPEN UNIVERSITY
SCHOOL OF COMPUTER AND INFORMATION SCIENCES
MAIDAN GARHI, NEW DELHI — 110068



BCA

(Bachelor of Computer Applications)

BCSP-064: PROJECT

TITLE OF PROJECT	Design and Development of an E-Commerce Shopping Website
NAME	KRISH
ENROLLMENT NUMBER	2250914981
PROGRAMME	BCA (Bachelor of Computer Applications)
COURSE CODE	BCSP-064
REGIONAL CENTRE CODE	07 — Delhi 1 Mohan Estate (South Delhi)
STUDY CENTRE CODE	07588 — NSIC Technical Service Center
E-MAIL ID	Krishgeek777@gmail.com
PHONE NUMBER	7011631350

 इंदिरा गांधी राष्ट्रीय मुक्त विश्वविद्यालय मैदान गढ़ी, नई दिल्ली - 110068 Indira Gandhi National Open University Maidan Garhi, New Delhi - 110068		
IGNOU - Student Identity Card		
Enrolment Number :	2250914981	 
RC Code :	07: DELHI 1 (MOHAN ESTATE (SOUTH DELHI))	
Name of the Programme :	BCA : BACHELOR OF COMPUTER APPLICATIONS	
Name :	KRISH	
Guardian's Name :	RAM KARAN SINGH	
Address :	BLOCK G, 9/213 Ratiya Marg SANGAM VIHAR New Delhi DELHI	
Pin Code :	110080	
Instructions :		
1. This card should be produced on demand at the Study Center, Examination Center or any other Establishment of IGNOU to use its facilities.		
2. The facilities would be available only relating to the Programme/course for which the student is registered.		
3. This ID Card is generated online. Students are advised to take a color print of this ID Card and get it laminated.		
4. The student details can be cross checked with the QR Code at www.ignou.ac.in . PD 2025-07-01		
		 Registrar Student Registration Division

IX CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled _____

Submitted to **Indira Gandhi National Open University** in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF COMPUTER APPLICATIONS (BCA)**, is an original work carried out by Mr./ Ms. _____

Enrolment No.: _____ under the guidance of Mr./ Ms. _____

The matter embodied in this project is a genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirement of any course of study.

Signature of the Student

Signature of the Guide

Name and Address
of the student :

Name, Designation and
Address of the Guide

Enrolment No.:

Guide Biodata

Name:

Mr. Kushagra

Career Objective

To secure a challenging role in the field of Information Technology where I can contribute my software development experience and engineering knowledge towards innovative solutions. I aim to foster growth, lead technology-driven initiatives, and mentor students and professionals toward excellence in the IT industry.

Technical Skills:

Category	Details
Languages/Technologies	JavaScript, Node.js, React.js, C++, Java
Database	MongoDB, MySQL
Frameworks	Express.js, REST APIs, Tailwind CSS, Bootstrap
Tools & Platforms	Git, GitHub, Postman, VS Code, Vercel, Docker
Operating Systems	Windows 10/11, Linux (Ubuntu), macOS

Professional Experience:

Company	Role	Duration
Infosys Ltd.	Software Engineer	Jan 2020 – Dec 2021
Coforge Ltd.	Full Stack Developer	Jan 2022 – Present

Roles & Responsibilities:

- Developed scalable web applications using MERN stack
- Collaborated in Agile teams to deliver enterprise-grade solutions
- Participated in code reviews and deployment pipelines
- Mentored interns and junior developers on real-time projects
- Managed REST API development and MongoDB integrations
- Conducted technical training sessions and client demos

 **भारत सरकार**
Government of India 

Issue Date: 08/06/2014



कुशाग्र
Kushagra
जन्म तिथि/DOB: 26/08/2000
पुरुष/ MALE

4814 2801 9167
VID : 9187 2404 5598 1901

मेरा आधार, मेरी पहचान

Synopsis

Title of the Project:**Design and Development of an E-Commerce Shopping Website****1. Introduction and Objectives of the Project****1.1 Introduction**

In the ever-evolving digital era, the phenomenon of online shopping has significantly reshaped consumer behavior and the global retail landscape. The E-commerce industry has seen exponential growth over the past decade, and it continues to evolve with the integration of new technologies. E-commerce provides a platform for businesses to offer a broad range of products and services to a global audience, eliminating geographical barriers and making shopping a convenient and seamless experience.

With the advancement of internet accessibility and smartphone usage, the modern consumer prefers the flexibility and ease of online shopping over traditional methods. This transformation necessitates the development of efficient, secure, and scalable E-commerce platforms that can cater to dynamic customer demands.

The primary goal of this project titled "**Design and Development of an E-Commerce Shopping Website**" is to create a fully functional, user-centric online shopping platform. This website will offer comprehensive features such as user authentication, product listings, search and filter functionality, shopping cart management, a secure checkout process, and an administrative dashboard for managing products and orders. The website is developed using the modern and efficient MERN stack (MongoDB, Express.js, React.js, and Node.js), which provides robustness, modularity, and scalability to the application.

1.2 Objectives

The key objectives of this project are as follows:

- To develop a responsive and intuitive front-end user interface using React.js.
- To design a robust and secure back-end using Node.js and Express.js for managing server-side operations and APIs.
- To implement user authentication and authorization features, including login, registration, and session management.

- To enable product listing, search, filtering, and detailed view functionalities.
- To provide a dynamic shopping cart system where users can add, update, or remove items.
- To integrate a secure checkout process with order storage.
- To build an admin panel that allows authorized personnel to manage inventory, view orders, and update product data.
- To ensure the application is scalable, secure, and performance-optimized.

2. Project Category

Category: Web-Based Application with RDBMS

The “E-Commerce Shopping Website” is classified as a **Web-Based Application** that utilizes a **Relational Database Management System (RDBMS)**, though implemented via **NoSQL (MongoDB)** — which still follows structured data modeling through collections and schemas. The application is developed using the **MERN stack** (MongoDB, Express.js, React.js, and Node.js), which is one of the most popular technology stacks for building full-stack web applications today.

It involves:

- Front-end development for user interaction (React.js)
- Back-end for business logic and API handling (Node.js & Express.js)
- Database layer to store and manage persistent data (MongoDB)
- RESTful architecture to separate client-server communication
- Admin interface for data and product management

2.1 Brief Introduction about RDBMS

A **Relational Database Management System (RDBMS)** is a type of database system that stores data in the form of related tables. It uses **structured rows and columns** to organize data and relies on **SQL (Structured Query Language)** for data manipulation and query operations. Each table in an RDBMS has a unique key (called a **Primary Key**) and may relate to other tables through **Foreign Keys**, ensuring **data integrity and referential consistency**.

RDBMS provides:

- **Data normalization** to reduce redundancy
- **Efficient query processing** using indexing and joins
- **ACID properties** (Atomicity, Consistency, Isolation, Durability) to ensure reliable transactions
- **Security features** like user roles, constraints, and data validation.

3. Problem Definition of Project

In today's fast-paced world, **traditional offline retail models** are struggling to meet the expectations of modern consumers who prefer quick, convenient, and accessible shopping experiences. While large companies have already transitioned to online platforms, **many small to medium-sized sellers** lack the technical knowledge and resources to create their own E-commerce solutions.

This project aims to solve the following problems:

1. **Lack of Centralized Platform:**
Many sellers do not have an online presence where they can list, manage, and sell their products.
2. **Manual Business Handling:**
Inventory, order tracking, and customer management are often done manually, leading to inefficiencies and errors.
3. **Limited Customer Access:**
Without an online system, customers are restricted by physical store hours and locations.
4. **No Personalized User Experience:**
Traditional models cannot offer features like user accounts, saved carts, or product recommendations.
5. **Security Concerns:**
Even those who attempt to go online often face challenges implementing **secure login systems** and **safe transaction flows**.

4. Module Description and Process Logic

The project is divided into **five core modules**, each handling a specific aspect of the application. These modules work together through a clearly defined process flow and RESTful APIs.

◇ 1. User Module

This module manages user authentication and profile data.

- **Features:**
 - User registration
 - Secure login/logout
 - Profile viewing/editing
- **Data Used:** Users collection in MongoDB
- **Security:** Passwords hashed using bcrypt, JWT tokens used for session control

◇ 2. Product Module

This module handles product-related operations for both users and admin.

- **Features:**
 - Product listing with pagination
 - Search and filter by category or keyword
 - Product detail page
- **Data Used:** Products collection
- **Access:** Readable by all users; writable by admin

◇ 3. Cart Module

This module lets users manage their shopping cart.

- **Features:**
 - Add or remove products
 - Update quantity
 - Calculate total price
- **Data Used:** Carts collection linked to each user
- **State:** Temporary (until checkout)

◇ 4. Order Module

Handles placing, saving, and retrieving orders.

- **Features:**
 - Checkout functionality

- Saving order data
- Order history and confirmation
- **Data Used:** Orders and Order_Items collections
- **Transaction:** Triggered after cart checkout

◇ 5. Admin Module

Available only to admin users for management tasks.

- **Features:**
 - Add/Edit/Delete products
 - View all orders
 - Dashboard with sales overview
- **Security:** Access controlled via role-based authentication.

Process Logic

The entire application follows a **modular, client-server architecture** using REST APIs and React components.

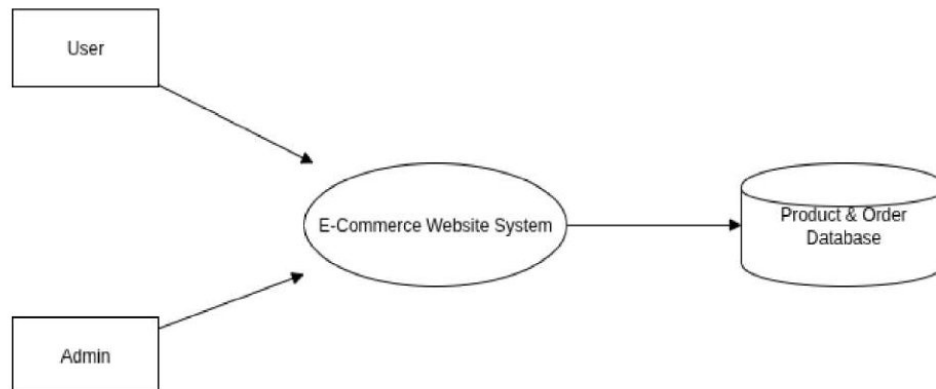
Step-by-Step Logical Flow:

1. **User Registration / Login**
 - a. User fills the form → API validates → data saved in DB
 - b. JWT Token issued on successful login
2. **Product Browsing & Filtering**
 - a. Frontend requests products → Backend sends filtered results from DB
3. **Add to Cart Logic**
 - a. User clicks “Add to Cart”
 - b. Cart API checks if item exists → updates or inserts accordingly
 - c. Total is recalculated in real-time
4. **Checkout & Order Placement**
 - a. Cart data + shipping info sent to Order API
 - b. Items are stored in the Orders collection
 - c. Confirmation screen shown
5. **Admin Logic**
 - a. Admin logs in
 - b. Uses protected API routes to manage product data
 - c. Can view customer orders with filtering

4. Analysis of Project

◇ 4.1 Data Flow Diagrams (DFDs)

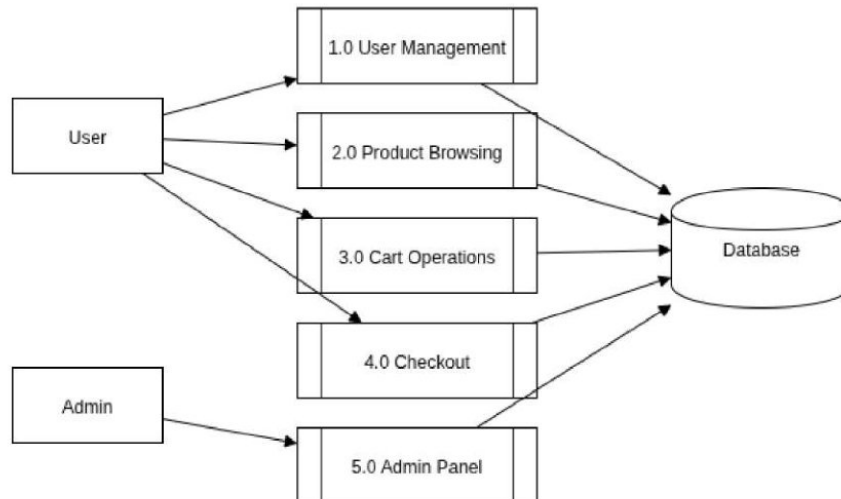
▤ DFD Level 0 (Context Diagram)



This diagram shows a high-level interaction between external entities (User, Admin) and the system.

- User ↔ E-Commerce Website ↔ Product Database
- Admin ↔ E-Commerce Website ↔ Order Database

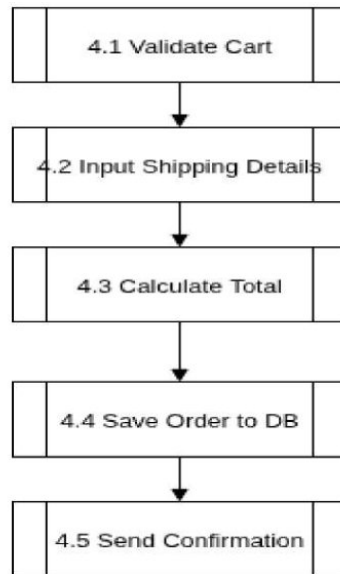
DFD Level 1 (Detailed System Flow)



Breaks the main system into sub-processes:

- 1.0: User Management
- 2.0: Product Viewing
- 3.0: Cart Processing
- 4.0: Checkout & Order
- 5.0: Admin Management

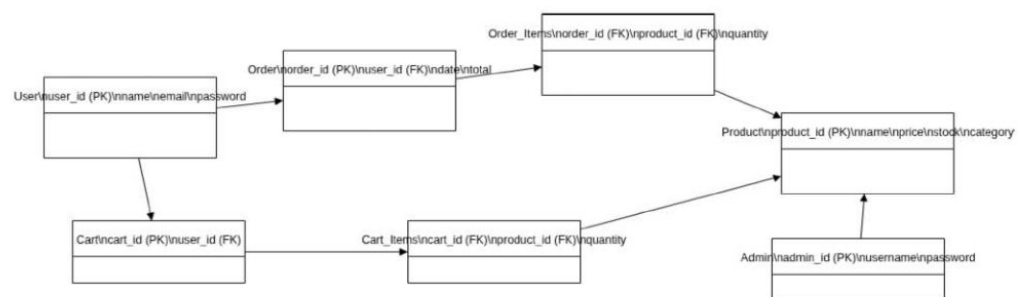
DFD Level 2



Shows internal steps of the Checkout Process:

- Validate Cart
- Capture Shipping Info
- Generate Invoice
- Save Order
- Send Confirmation

4.2 ER (Entity Relationship) Diagram



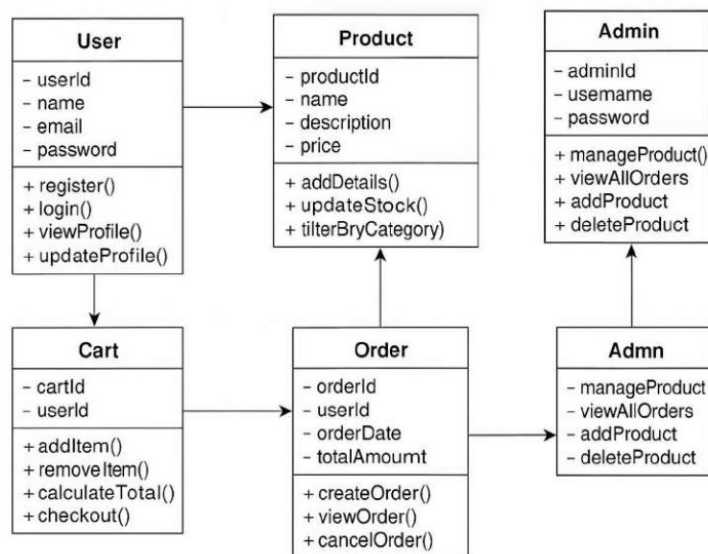
The ER diagram identifies all core entities in the system and shows their relationships:

- User → Order (1:M)
- Order → Product via Order_Items (M:N)
- User → Cart (1:1)
- Cart → Product via Cart_Items (M:N)

Entities:

- User(user_id, name, email, password)
- Product(product_id, name, price, category, stock)
- Order(order_id, user_id, date, total)
- Order_Items(order_id, product_id, quantity)
- Cart(cart_id, user_id)
- Cart_Items(cart_id, product_id, quantity)

4.3 Class Diagram (Object-Oriented View)



The **Class Diagram** is a part of the object-oriented design of the E-commerce system. It represents the **static structure** of the system using **classes, their attributes, methods (behaviors), and relationships**.

Main Classes & Description:

◊ *User*

- **Attributes:**
 - userId
 - name
 - email
 - password
- **Methods:**
 - register()
 - login()
 - viewProfile()
 - updateProfile()

◊ *Product*

- **Attributes:**
 - productId
 - name
 - description
 - price
 - stock
- **Methods:**
 - getDetails()
 - updateStock()
 - filterByCategory()

◊ *Cart*

- **Attributes:**
 - cartId
 - userId
 - items[]

- **Methods:**

- addItem()
- removeItem()
- calculateTotal()
- checkout()

◇ *Order*

- **Attributes:**

- orderId
- userId
- orderDate
- totalAmount

- **Methods:**

- createOrder()
- viewOrder()
- cancelOrder()

◇ *Admin*

- **Attributes:**

- adminId
- username
- password

- **Methods:**

- manageProduct()
- viewAllOrders()
- addProduct()
- deleteProduct()

Relationships Between Classes:

- User **has** one Cart (1:1)
- User **places** many Orders (1:M)
- Cart **contains** many Products (M:N)
- Order **contains** many Products (M:N)
- Admin **manages** many Products (1:M)

5. Data Structures

1. User Table

Field Name	Data Type	Description
user_id	String/Int (PK)	Unique ID for each user
username	String	Name of the user
email	String	User's email address
password	String (Hashed)	User's login password
contact_number	String	User's phone number
address	String	Shipping address
user_role	String	Role (admin/customer)

2. Product Table

Field Name	Data Type	Description
product_id	String/Int (PK)	Unique product identifier
product_name	String	Name of the product
description	String	Details about the product
price	Float	Price of the product
category	String	Product category (e.g., Electronics)
stock_quantity	Int	Available stock
product_image	Array/String	Image URLs or paths

3. Order Table.

Field Name	Data Type	Description
order_id	String/Int (PK)	Unique order ID
user_id	String (FK)	ID of the user who placed the order
product_list	Array/Object	List of product IDs with quantities
total_amount	Float	Total order cost
order_status	String	Status (pending/shipped/delivered)
order_date	DateTime	Date of order placement
shipping_addresses	String	Address to deliver the order

4. Cart Table

Field-Name	Data Type	Description
cart_id	String/Int (PK)	Unique cart ID
user_id	String (FK)	User ID for whom cart is maintained
product_list	Array/Object	Items in cart with quantity
total_price	Float	Total cart value

5. Payment Table

Field Name	Data Type	Description
payment_id	String/Int (PK)	Unique payment ID
order_id	String (FK)	Linked order ID
payment_method	String	Mode of payment (e.g., UPI, Card)
payment_status	String	Status (success/failure/pending)
transaction_id	String	Transaction reference number
payment_date	DateTime	Date and time of payment

System Requirements

1. Hardware Requirements

Component	Minimum Requirement
Processor	Intel i3 or equivalent (Dual Core)
RAM	4 GB (8 GB Recommended)
Hard Disk	100 GB free space
Monitor	15" color monitor
Keyboard & Mouse	Standard input devices
Internet	Broadband connection (for live testing)

2. Software Requirements

Component	Specification / Tools
Operating System	Windows 10 / Linux / macOS
Frontend Tools	HTML, CSS, JavaScript, React (optional)
Backend Tools	Node.js, Express.js
Database	MongoDB or MySQL
Web Server	Node server / Apache (for deployment)

Browser	Google Chrome, Mozilla Firefox, etc.
IDE / Editor	VS Code, Sublime Text
Version Control	Git and GitHub
Payment Gateway	Razorpay, PayPal, or Stripe

6. System Development Life Cycle (SDLC)

The development of the **E-commerce Shopping Website** follows the traditional phases of the **System Development Life Cycle (SDLC)** to ensure systematic planning, development, and deployment.

1. Feasibility Study

In this phase, the feasibility of developing the system is analyzed from various perspectives:

- **Technical Feasibility:** Evaluates whether the current technology stack (Node.js, MongoDB, etc.) can support the project.
- **Economic Feasibility:** Ensures the project can be built within budget using open-source tools.
- **Operational Feasibility:** Determines whether the website will be user-friendly and effectively serve its intended purpose.

2. System Analysis

This phase involves gathering detailed requirements from the users and stakeholders. It focuses on understanding what features the system must include such as:

- User registration and login
- Product search and browsing
- Cart management
- Payment processing
- Order tracking

3. System Design

In this phase, the technical architecture and system design are created based on the analysis:

- **Frontend Design:** Page layout, navigation, responsiveness.
- **Backend Design:** API structure, data flow, business logic.
- **Database Design:** Tables or collections for users, products, orders, and payments.

4. Implementation (Coding)

This is the phase where the actual development takes place. The website is built using the chosen technologies:

- Frontend: HTML, CSS, JavaScript, React (optional)
- Backend: Node.js, Express.js
- Database: MongoDB or MySQL

5. Testing

Testing ensures that the system is reliable, secure, and works as expected. Two primary types of testing used in this project are:

Black Box Testing

- Focuses on **functionality** of the application without looking into the internal code.
- Tester does not need knowledge of programming.
- Example tests:
 - Login with correct and incorrect credentials
 - Add to cart and checkout flow
 - Payment success/failure scenarios
 - Order tracking for different statuses

White Box Testing

- Involves testing the **internal logic** and structure of the code.
- Tester must have knowledge of the source code.

- Example tests:
 - Unit testing individual functions or modules
 - Checking conditional statements, loops, error handling in backend code
 - API request/response handling and validation

These testing types are applied together to ensure both **external behavior** and **internal correctness** of the system.

6. Deployment

Once tested, the application is deployed on a web server so users can access it live. Hosting platforms like Vercel or Netlify may be used for frontend, and Render or Railway for backend services.

7. Industry–Client Interface

This project, “**Design and Development of an E-Commerce Shopping Website,**” has been undertaken as an academic requirement for the completion of the Bachelor of Computer Applications (BCA) program under **Indira Gandhi National Open University (IGNOU)**.

There is **no external industry or client** involved in this project. However, the project is modeled to simulate a **real-world scenario** where a developer is tasked with creating a scalable and functional E-commerce system for a small-to-medium business.

7.1 Assumed Client Description:

To keep the development aligned with industry standards, the system is designed assuming a **fictitious retail client** who wishes to launch an online shopping platform to sell physical products such as electronics, fashion, or groceries. The application fulfills all typical client needs like:

- Secure user login and product management
- Dynamic product catalog
- Order and cart management
- Admin control panel
- Inventory visibility and reports

◇ Relevance to Industry:

Even though there is no official industry partner, the project incorporates:

- **Software engineering practices** (modular development, version control, testing)
- **Industry-used technologies** like MERN stack (MongoDB, Express.js, React.js, Node.js)
- **User and admin feature** common in actual commercial E-commerce applications

This approach ensures that the student not only completes the project academically but also builds a portfolio-worthy project that is relevant and deployable in a real-world business setting.

8. Future Scope of the Project

The current version of the E-Commerce Shopping Website is designed as a functional prototype with all major features expected from a commercial online store. However, it holds immense potential for future enhancement and scalability.

Future Enhancements:

- **Integration with Online Payment Gateways:**
Implement real-time payment systems like Razorpay, PayPal, or Stripe for secure transactions.
- **AI-Powered Recommendation Engine:**
Use machine learning to analyze user behavior and recommend personalized products.
- **Mobile App Development:**
Create a cross-platform mobile app version using React Native or Flutter.
- **Multi-vendor Functionality:**
Allow multiple sellers to register and manage their own product listings.
- **Customer Support Chatbot:**
Integrate chatbot or live chat for 24/7 user assistance.
- **Invoice Generation & GST Support:**
Auto-generate downloadable invoices with tax calculations.
- **Feedback and Ratings System:**
Let users review products and share their experiences.

These improvements would transform the prototype into a full-scale, production-ready product suited for commercial deployment.

8. Bibliography / References

- [MDN Web Docs](#)
- React Official Documentation
- Node.js Documentation
- [MongoDB Documentation](#)
- [Express.js Documentation](#)
- IGNOU Project Guidelines for BCSP-064
- TutorialsPoint & GeeksforGeeks (for conceptual learning)

Thank You

I sincerely thank the **Indira Gandhi National Open University (IGNOU)** for providing this opportunity to apply practical knowledge in the form of a project. This experience has enhanced my understanding of software development, system design, and real-world implementation.

A special thanks to my project guide and the academic coordinators for their support and feedback during this journey.

Project Report

CHAPTER 1: INTRODUCTION

1.1 Background

E-commerce, short for electronic commerce, refers to the buying and selling of goods and services over the internet. In recent years, online shopping has experienced explosive growth worldwide, especially in India. According to industry reports, the Indian e-commerce market is expected to reach \$200 billion by 2026, driven by increasing internet penetration, smartphone usage, and digital payment adoption.

Traditional brick-and-mortar stores have limitations such as geographical constraints, fixed operating hours, and limited product variety. E-commerce platforms overcome these limitations by providing 24/7 availability, a wider product range, convenient home delivery, and competitive pricing.

The development of modern web technologies such as React.js, Node.js, Express.js, and MongoDB (collectively known as the MERN Stack) has made it possible to build powerful, scalable, and responsive web applications efficiently. The MERN Stack enables full-stack JavaScript development, allowing developers to use a single programming language across both the client-side and server-side.

This project, titled “**Design and Development of an E-Commerce Shopping Website**”, aims to build a fully functional e-commerce platform named **ShopKart** using the MERN Stack. ShopKart provides users with a seamless online shopping experience, from browsing products to placing orders, along with a comprehensive admin panel for managing products and orders.

2.1 Scope

The scope of this project covers the design and development of a web-based e-commerce shopping website named ShopKart using the MERN Stack. The application provides a complete online shopping experience for customers and a management interface for administrators.

The project includes the following modules and features:

User Module:

- User registration with name, email, and password
- Secure user login with JWT (JSON Web Token) authentication
- Password hashing using bcrypt algorithm for data security
- User profile management and session handling

Product Module:

- Browse all products with images, descriptions, prices, and ratings
- Search products by name (case-insensitive partial matching)
- Filter products by category (Electronics, Clothing, Books, Home, Sports, Beauty)
- Sort products by price (low to high, high to low), rating, and newest first
- View detailed product information with stock availability

Cart Module:

- Add products to shopping cart with quantity selection
- Update product quantity in cart (increment/decrement)
- Remove individual items from the cart
- View cart total with order summary (subtotal, delivery, total)

Order Module:

- Checkout with shipping address (street, city, state, pincode)
- Payment method selection (Cash on Delivery)
- Order placement with automatic stock reduction and cart clearing
- Order history with status tracking (Pending, Processing, Shipped, Delivered)

Admin Module:

- Admin dashboard with statistics (total products, orders, revenue)
- Add new products with image upload using Multer
- Edit and delete existing products
- View all customer orders with details
- Update order status (Pending → Processing → Shipped → Delivered)

Features not included in the current scope (can be added in future):

- Online payment gateway integration (Razorpay, Stripe)
- Email/SMS notifications for order updates
- Product reviews and ratings by customers
- Wish list functionality
- Mobile application (iOS/Android)

2.2 Objective

The primary objectives of this project are:

1. To design and develop a full-stack e-commerce web application using the MERN Stack (MongoDB, Express.js, React.js, Node.js).
2. To implement secure user authentication using JSON Web Tokens (JWT) and password hashing with bcrypt algorithm.
3. To provide essential e-commerce functionality including product browsing with search, category filtering, and sorting options.
4. To develop a shopping cart system that allows users to add, update, and remove items before proceeding to checkout.
5. To implement an order processing system with shipping address, payment method selection, and order status tracking.
6. To build an admin dashboard for managing products (CRUD operations with image upload) and orders (status updates).
7. To create RESTful APIs following the MVC (Model-View-Controller) architecture for clean and maintainable code organization.
8. To ensure data security through password hashing, JWT-based route protection, and role-based access control (user and admin roles).
9. To develop a responsive and user-friendly interface using React.js and Tailwind CSS that works across desktop and mobile devices.
10. To follow the Software Development Life Cycle (SDLC) methodology and document all phases including analysis, design, implementation, and testing.

3. SOFTWARE REQUIREMENT SPECIFICATION

1. Introduction

1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to provide a detailed description of the requirements for the ShopKart E-Commerce Shopping Website. This

document describes the intended features, interfaces, and constraints of the system to serve as a reference for the development process.

This SRS document is intended for the following audience:

- Developers — to understand what needs to be built
- Project Guide — to evaluate the project scope and requirements
- Evaluators — to verify that the system meets the specified requirements
- Users — to understand the system's capabilities and limitations

1.2 Definitions and Acronyms

The following definitions and acronyms are used throughout this document:

Term	Definition
MERN	MongoDB, Express.js, React.js, Node.js — a full-stack JavaScript framework
JWT	JSON Web Token — a compact, URL-safe token format used for authentication
API	Application Programming Interface — a set of rules for software communication
REST	Representational State Transfer — an architectural style for designing APIs
CRUD	Create, Read, Update, Delete — basic database operations
MVC	Model-View-Controller — a software design pattern
SRS	Software Requirement Specification
ODM	Object Data Modeling — Mongoose library for MongoDB
COD	Cash on Delivery — a payment method
UI	User Interface — the visual part of the application
SDLC	Software Development Life Cycle
HTTP	Hypertext Transfer Protocol — communication protocol for the web
bcrypt	A password hashing algorithm based on the Blowfish cipher
npm	Node Package Manager — tool for managing JavaScript packages
MongoDB	A NoSQL document-oriented database
Express.js	A minimal and flexible Node.js web application framework

React.js	A JavaScript library for building user interfaces by Facebook
Node.js	A JavaScript runtime built on Chrome's V8 engine
Tailwind CSS	A utility-first CSS framework for rapid UI development
Axios	A promise-based HTTP client for making API requests
Multer	A Node.js middleware for handling file uploads
Mongoose	An ODM library for MongoDB and Node.js

1.3 Overview

ShopKart is a web-based e-commerce shopping website developed using the MERN Stack. The system provides a platform for customers to browse, search, and purchase products online, while administrators can manage the product catalog and process orders.

The system consists of three main tiers:

1. Presentation Tier (Frontend): Built with React.js and Tailwind CSS. Handles the user interface, client-side routing, state management, and API communication using Axios.

2. Application Tier (Backend): Built with Node.js and Express.js. Implements REST APIs following the MVC pattern with controllers, routes, and middleware for authentication and authorization.

3. Data Tier (Database): MongoDB Atlas (cloud-hosted NoSQL database) with Mongoose ODM for schema definition and data validation. Contains four collections: Users, Products, Cart, and Orders.

The remaining sections of this SRS document describe the general description of the product, specific functional and non-functional requirements, external interfaces, performance requirements, design constraints, and software system attributes.

2. General Description

2.1 Product Perspective

ShopKart is a standalone, self-contained web application that provides a complete e-commerce shopping experience. It is not a part of any larger system; rather, it is an independent product designed to operate on its own. The application follows a client-server architecture where the frontend and backend are separate applications that communicate through REST APIs.

The system operates in the following environment:

- The frontend (React.js application) runs in the user's web browser (Chrome, Firefox, Edge)
- The backend (Node.js + Express.js server) runs on a server listening on port 5000
- The database (MongoDB Atlas) is hosted on the cloud, accessible via the internet
- Images uploaded by admin are stored locally on the server in the uploads/ directory
- Seeded product images are sourced from public URLs (Unsplash)

The frontend communicates with the backend using Axios HTTP client. All API requests to protected routes include a JWT token in the Authorization header. The backend validates the token using middleware before processing the request. This separation of concerns ensures that the frontend and backend can be developed, tested, and deployed independently.

The application supports two user roles: Customer (default role assigned during registration) and Admin (assigned manually in the database). Role-based access control ensures that only admin users can access the admin dashboard and management features.

2.2 Product Functions

The major functions of ShopKart e-commerce system are as follows:

User Authentication Functions:

- **Register:** Register —

New users can create an account by providing their name, email address, and password. The password is hashed using bcrypt (10 salt rounds) before storing in the database. A JWT token is generated and returned to the client for session management.

- **Login:** Login —

Existing users can login using their email and password. The system compares the entered password with the stored hash using `bcrypt.compare()`. Upon successful authentication, a JWT token with 7-day expiry is returned.

Product Browsing Functions:

- View all products in a responsive grid layout with product cards
- Search products by name using case-insensitive regex matching
- Filter products by category: Electronics, Clothing, Books, Home, Sports, Beauty, Other
- Sort products by: Price (Low to High), Price (High to Low), Rating, Newest First
- View detailed product page with image, description, ratings, stock, and quantity selector

Cart Management Functions:

- Add a product to the cart with a specified quantity
- If the product already exists in cart, increment its quantity automatically
- Update the quantity of any cart item using +/- buttons
- Remove individual items from the cart
- View order summary with subtotal, delivery charges (free), and total amount

Order Processing Functions:

- Checkout by entering shipping address (street, city, state, pincode)
- Select payment method (Cash on Delivery)
- Place order — system creates order, reduces product stock, and clears cart
- View order history with order ID, date, total, payment method, and status badge

Admin Management Functions:

- View dashboard with total products count, total orders count, and total revenue
- Add new products with name, description, price, category, stock, and image upload
- Edit existing product details
- Delete products from the catalog
- View all customer orders with details
- Update order status: Pending → Processing → Shipped → Delivered

2.3 User Characteristics

The system has two types of users:

1. Customer (User Role)

Customers are general internet users who wish to shop for products online. They are expected to have basic knowledge of web browsing, filling online forms, and navigating websites. Customers familiar with existing e-commerce platforms such as Amazon, Flipkart, or Myntra will find the ShopKart interface intuitive and familiar. Customers can register a new account, browse products, manage their cart, place orders, and view their order history.

2. Administrator (Admin Role)

Administrators are store managers or website operators responsible for managing the e-commerce platform. They are expected to have knowledge of product management, inventory control, and order fulfillment processes. Admin users can access the admin dashboard, add or edit products with image uploads, delete products, view all customer orders, and update order statuses. The admin role is assigned manually in the database by setting the user's role field to "admin".

2.4 Constraints

The following constraints apply to the ShopKart system:

- The application requires an active internet connection to access the MongoDB Atlas cloud database. Without internet, the backend cannot connect to the database.
- The system currently supports only Cash on Delivery (COD) as the payment method. Online payment gateways (Razorpay, Stripe, UPI) are not integrated in the current version.

- File uploads are limited to image files (JPEG, PNG, WebP) for product images. The maximum file size is determined by the Multer middleware configuration.
- Product images uploaded by admin are stored locally on the server in the uploads/ directory. In a production environment, cloud storage (AWS S3, Cloudinary) would be preferred.
- The admin role cannot be assigned through the user interface. It must be set directly in the MongoDB database by changing the user's role field from "user" to "admin".
- The application is designed for desktop and laptop screens with a minimum resolution of 1366×768. While it is responsive, it has not been extensively tested on all mobile devices.
- The system does not implement real-time features. Order status updates are not pushed to users in real-time; users must refresh the page to see the latest status.
- The JWT token has a fixed expiry of 7 days. Users will be automatically logged out and redirected to the login page when the token expires.

3. Specific Requirements

3.1 External Interface Requirements

This section describes all the external interfaces of the ShopKart system, including user interfaces, hardware interfaces, software interfaces, and communication interfaces.

3.1.1 User Interfaces

The ShopKart application provides a modern, responsive, and user-friendly web interface built using React.js and styled with Tailwind CSS. The interface follows a consistent design language across all pages with a navigation bar at the top and a footer at the bottom. Below is a description of each user interface screen:

Home Page: The home page is the main landing page of ShopKart. It features a hero section with a gradient banner displaying the website tagline. Below the hero section, there is a product grid displaying all available products in a 4-column responsive layout. Each product is shown as a card with its image, category badge, name, star rating, price in INR (₹), and an "Add to Cart" button. The page includes category filter pills (All, Electronics,

Clothing, Books, Home, Sports, Beauty, Other) and a sort dropdown (Newest First, Price Low-High, Price High-Low, Rating). A search bar is integrated into the navigation bar.

Login Page: The login page presents a centered form with two input fields — Email Address and Password. The password field includes a show/hide toggle button for user convenience. A "Sign In" button submits the form. Below the form, there is a link saying "Don't have an account? Register" that navigates to the registration page. Error messages are displayed as toast notifications using React Hot Toast.

Register Page: The registration page contains a form with four input fields — Full Name, Email Address, Password, and Confirm Password. The password fields include show/hide toggle buttons. A "Create Account" button submits the form after validating that all fields are filled and passwords match. A link to the login page is provided at the bottom.

Product Details Page: This page displays detailed information about a single product. It shows a large product image on the left side and product details on the right side, including: product name, category, star rating with numeric value, price in INR, stock availability status (In Stock / Out of Stock), a quantity selector with +/- buttons, an "Add to Cart" button, and the full product description.

Cart Page: The cart page displays all items currently in the user's shopping cart. Each item shows the product image (thumbnail), product name, unit price, quantity controls (+/- buttons), item total, and a delete (trash) icon to remove the item. On the right side, an Order Summary card displays the subtotal, delivery charges (Free for orders above ₹500), and the total amount. A "Proceed to Checkout" button navigates to the checkout page. If the cart is empty, a message is shown with a "Continue Shopping" link.

Checkout Page: The checkout page is divided into two sections. The left section contains the Shipping Address form with four input fields: Street Address, City, State, and Pincode. Below the address form is the Payment Method selection (currently Cash on Delivery only) and a "Place Order" button. The right section shows a summary of all items being ordered with thumbnails, names, quantities, and prices, along with the total amount.

Order History Page: This page displays all past orders of the logged-in user. Each order is shown in a card format with the order ID (last 8 characters), order date, total amount, payment method, and a color-coded status badge (Pending: yellow, Processing: blue, Shipped: purple, Delivered: green). Expanding an order shows item-wise breakdown with product images, names, quantity $\times$ price, and the shipping address.

Admin Dashboard: The admin dashboard provides an overview of the store's performance. It displays three statistics cards: Total Products, Total Orders, and Total Revenue (₹). Below the cards, a table shows recent orders with columns for Order ID, Customer Name, Amount, Date, and Status. Navigation links to Manage Products and Manage Orders pages are provided.

Admin — Manage Products: This page displays all products in a table format with columns for Image (thumbnail), Product Name, Category, Price, Stock, and Actions (Edit/Delete buttons). An "Add Product" button opens a modal form with fields for Name, Description, Price, Stock, Category (dropdown), and Image Upload. The same modal is used for editing existing products.

Admin — Manage Orders: This page shows all customer orders in card format. Each card displays the order ID, customer name, order date, total amount, item count, and a status dropdown. The admin can change the order status by selecting a new value from the dropdown (Pending, Processing, Shipped, Delivered). The status is updated immediately via an API call.

3.1.2 Hardware Interface

The ShopKart application is a web-based system and does not interact directly with any specific hardware components. It runs on standard computer systems with web browser capability. The hardware requirements are as follows:

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent (2 GHz+)
RAM	4 GB (8 GB recommended for development)
Hard Disk	256 GB SSD (for Node.js, npm packages, and project files)
Internet	Broadband connection (for MongoDB Atlas cloud database)

Display	Monitor with 1366 × 768 resolution minimum
Input Devices	Standard keyboard and mouse/trackpad

For the development environment, a system with at least 8 GB RAM is recommended to run the Node.js backend server, React.js development server (Vite), VS Code editor, and web browser simultaneously without performance issues.

3.1.3 Software Interface

The following software components are used in the ShopKart system:

Software	Version	Purpose
Node.js	v18+	JavaScript runtime for backend server
Express.js	v5	Web application framework for REST APIs
React.js	v19	Frontend UI library for building components
MongoDB Atlas	Cloud	NoSQL database for data storage
Mongoose	v9	ODM library for MongoDB schema & validation
Tailwind CSS	v3	Utility-first CSS framework for styling
Vite	v8	Frontend build tool and development server
jsonwebtoken	v9	JWT token generation and verification
bcryptjs	v3	Password hashing algorithm
Axios	Latest	HTTP client for API communication
React Router DOM	v6	Client-side routing and navigation
Multer	v2	Middleware for handling file uploads
React Hot Toast	Latest	Toast notification library
React Icons	Latest	Icon library (HeroIcons set)
Visual Studio Code	Latest	Code editor for development
Postman	Latest	API testing and debugging tool
Google Chrome	Latest	Primary web browser for testing

Git	Latest	Version control system
-----	--------	------------------------

3.1.4 Communication Interface

The ShopKart system uses HTTP (Hypertext Transfer Protocol) for all communication between the frontend client and the backend server. The communication follows the REST (Representational State Transfer) architectural style.

The communication interfaces are described below:

API Base URL: `http://localhost:5000/api`

Request Headers: All API requests include the following headers:

- Content-Type: application/json — for sending and receiving JSON data
- Content-Type: multipart/form-data — for image upload requests (Multer)
- Authorization: Bearer {JWT_TOKEN} — for authenticated/protected routes

HTTP Methods Used:

- GET — Retrieve data (products, cart, orders, profile)
- POST — Create new data (register, login, add to cart, place order, add product)
- PUT — Update existing data (update cart quantity, update product, update order status)
- DELETE — Remove data (remove cart item, clear cart, delete product)

Response Format: All API responses are in JSON format with the following structure:

- Success: { success: true, data: {...}, message: "..." }
- Error: { success: false, message: "Error description" }

Status Codes:

- 200 — OK (successful request)
- 201 — Created (resource created successfully)
- 400 — Bad Request (invalid input or validation error)
- 401 — Unauthorized (missing or invalid JWT token)

- 403 — Forbidden (insufficient permissions, admin-only route)
- 404 — Not Found (resource does not exist)
- 500 — Internal Server Error (server-side error)

The frontend uses an Axios instance (axiosInstance.js) configured with the base URL and request/response interceptors. The request interceptor automatically attaches the JWT token from localStorage to every outgoing request. The response interceptor catches 401 errors (expired token) and redirects the user to the login page.

3.2 Functional Requirements

This section describes the functional requirements of the ShopKart system. Each operation is described with its input, processing, and expected output.

3.2.1 Login Operation

The login operation allows registered users to authenticate and access protected features.

- **Input:** Email address and password entered by the user on the Login page.
- **Processing:**
 - The system receives the email and password via POST /api/auth/login
 - The system searches for a user with the given email in the Users collection
 - If no user is found, an error message "Invalid email or password" is returned
 - If user is found, the entered password is compared with the stored hashed password using bcrypt.compare()
 - If password does not match, an error message "Invalid email or password" is returned
 - If password matches, a JWT token is generated with user's ID and role, with 7-day expiry
- **Output:** JSON response containing the JWT token, user name, email, and role. The token is stored in the browser's localStorage for subsequent API calls.

3.2.2 Search Operation

The search operation allows users to find products by name, category, or sorting preference.

- **Input:** Search query (text), category filter (dropdown), sort option (dropdown).
- **Processing:**
 - The system receives query parameters via GET
/api/products?search=...&category=...&sort=...
 - If search query is provided, the system uses MongoDB regex with case-insensitive flag (\$regex, \$options: 'i') to match product names
 - If category is provided, the system filters products where category matches exactly
 - If sort is provided, the system sorts results: price_asc (price ascending), price_desc (price descending), rating (rating descending), newest (createdAt descending)
 - Multiple filters can be combined (e.g., search + category + sort)
- **Output:** JSON array of matching products with all fields (name, description, price, category, image, stock, ratings). The total count of matching products is also returned.

3.2.3 Cart Operation

The cart operations allow authenticated users to manage their shopping cart.

- **Add to Cart — Input:** Product ID and quantity.
- **Processing:**
 - The system checks if the user already has a cart document in the Cart collection
 - If no cart exists, a new cart document is created with the product and quantity
 - If cart exists, the system checks if the product is already in the cart
 - If product exists in cart, its quantity is incremented by the specified amount
 - If product does not exist, it is added as a new item in the items array
- **Output:** Updated cart object with all items and quantities.
- **Update Quantity — Input:** Cart item ID and new quantity.
- **Processing:** The system finds the item in the cart and updates its quantity to the new value. Minimum quantity is 1.
- **Output:** Updated cart object.
- **Remove Item — Input:** Cart item ID.

- **Processing:** The system removes the specified item from the cart's items array using MongoDB \$pull operator.
- **Output:** Updated cart object without the removed item.

3.2.4 Checkout Operation

The checkout operation allows users to place an order from their cart.

- **Input:** Shipping address (street, city, state, pincode) and payment method (COD).
- **Processing:**
 - The system fetches the user's cart and populates product details using Mongoose populate()
 - If the cart is empty, an error "Your cart is empty" is returned
 - The system iterates through each cart item and:
 - — Checks if the product exists and has sufficient stock
 - — Builds the order items array with productId, quantity, and price at time of order
 - — Calculates the total amount (sum of price × quantity for each item)
 - — Reduces the product stock by the ordered quantity
 - A new Order document is created with all items, total amount, shipping address, payment method, and status set to "Pending"
 - The user's cart is cleared (items array set to empty)
- **Output:** JSON response with the created order details including order ID, items, total amount, shipping address, and status. User is redirected to Order History page.

3.3 Performance Requirements

The following performance requirements are defined for the ShopKart system:

- The frontend pages shall load within 3 seconds on a standard broadband connection (10 Mbps or above).
- The backend API endpoints shall respond within 500 milliseconds for standard CRUD operations.

- The system shall support at least 50 concurrent users without noticeable performance degradation.
- Product images shall be optimized for web display to minimize page load time.
- The React.js frontend shall use code splitting and lazy loading to reduce the initial bundle size.
- MongoDB queries shall use indexing on frequently searched fields (email, category) for faster retrieval.
- The Vite build tool shall generate a production-optimized bundle with minification and tree-shaking.

3.4 Design Constraints

The following design constraints were applied during the development of ShopKart:

- The application must be developed using the MERN Stack (MongoDB, Express.js, React.js, Node.js) as specified in the project proposal.
- The backend must follow the MVC (Model-View-Controller) architecture pattern for clean separation of concerns.
- All backend APIs must follow RESTful conventions (proper HTTP methods, status codes, and URL naming).
- The frontend must be responsive and work on screens with minimum 1366×768 resolution.
- User passwords must never be stored in plain text; bcrypt hashing with a minimum of 10 salt rounds is required.
- All protected API routes must use JWT middleware for authentication before processing requests.
- The application must use environment variables (.env file) for sensitive configuration like database URI and JWT secret.
- The project code must be version-controlled using Git with a clean commit history.

3.5 Logical Database Requirements

The ShopKart system uses MongoDB, a NoSQL document-oriented database, hosted on MongoDB Atlas (cloud). The database is named "shopkart" and consists of four collections:

Collection	Purpose	Key Fields
users	Stores registered user data	_id, name, email, password (hashed), role, createdAt
products	Stores product catalog	_id, name, description, price, category, image, stock, ratings
carts	Stores shopping cart per user	_id, userId (FK), items [{productId, quantity}]
orders	Stores placed orders	_id, userId (FK), items [{productId, qty, price}], totalAmount, shippingAddress, status, paymentMethod

The relationships between collections are managed using Mongoose references (ObjectId). The userId field in Cart and Order collections references the Users collection. The productId field in cart items and order items references the Products collection. Mongoose populate() is used to fetch referenced documents when needed (e.g., displaying product names and images in the cart and order pages).

The detailed database schema with all fields, data types, constraints, and relationships is provided in Chapter 6: Analysis of Project (Section 6.5 — Data Structure).

3.6 Software System Attributes Requirements

3.6.1 Reliability

Reliability refers to the system's ability to perform its required functions under stated conditions for a specified period of time. The following measures ensure reliability in ShopKart:

- All backend controller functions use try-catch blocks to handle errors gracefully. If an error occurs, the system returns a meaningful error message instead of crashing.
- Input validation is performed on both frontend (React form validation) and backend (Mongoose schema validation) to prevent invalid data from entering the database.

- MongoDB Atlas provides automatic failover with replica sets. If the primary node fails, a secondary node takes over automatically, ensuring database availability.
- JWT tokens are verified on every protected API request. Invalid or expired tokens are rejected with a 401 status code, preventing unauthorized access.
- The bcrypt password hashing ensures that even if the database is compromised, user passwords cannot be reversed to their original form.

3.6.2 Availability

Availability refers to the proportion of time that the system is functional and accessible.

The following measures ensure availability in ShopKart:

- MongoDB Atlas cloud database provides 99.95% uptime SLA (Service Level Agreement) with data replication across multiple servers.
- The web application is accessible 24/7 from any device with a web browser and internet connection when deployed to a hosting platform.
- User sessions are maintained through JWT tokens stored in localStorage. Users remain logged in even after closing and reopening the browser, until the token expires (7 days).
- The React.js frontend is a Single Page Application (SPA) that loads once and handles navigation client-side, reducing server load and improving availability.
- Static assets (CSS, JavaScript bundles, images) can be cached by the browser, reducing load on the server for repeated visits.

3.6.3 Maintainability

Maintainability refers to the ease with which the system can be modified, updated, or repaired.

The following practices ensure maintainability in ShopKart:

- The backend follows the MVC architecture with separate folders for Models, Controllers, Routes, and Middleware. This separation of concerns makes it easy to locate and modify specific functionality.
- The frontend uses React's component-based architecture. Each page and UI element is a separate, reusable component that can be modified independently without affecting other parts.

- Context API (AuthContext, CartContext) centralizes state management, making it easy to update authentication or cart logic in one place.
- Environment variables (.env file) are used for configuration (database URI, JWT secret, port), allowing easy changes without modifying source code.
- The Axios instance is configured in a single file (axiosInstance.js), making it easy to change the API base URL or add global interceptors.
- Consistent coding style with ES6+ syntax (arrow functions, async/await, destructuring) improves code readability and maintainability.
- RESTful API design with descriptive endpoint names (e.g., /api/products, /api/cart/add, /api/orders/place) makes the API self-documenting and easy to understand.

4. TOOLS / ENVIRONMENT REQUIRED

This chapter describes the hardware and software tools required for the development and execution of the ShopKart E-Commerce Shopping Website. All tools used in this project are open-source or freely available, ensuring zero development cost.

Hardware Requirements

The minimum hardware configuration required to develop and run ShopKart is:

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 (2 GHz)	Intel Core i5 or above
RAM	4 GB	8 GB or above
Hard Disk	256 GB	512 GB SSD
Internet	Broadband (5 Mbps)	Broadband (10 Mbps+)
Display	1366 × 768	1920 × 1080 (Full HD)
Input Devices	Keyboard + Mouse	Keyboard + Mouse

Software Requirements

The following software tools and technologies are used in the development of ShopKart:

Software	Version	Purpose
Operating System	Windows 10/11 or Ubuntu Linux	Development platform
Node.js	v18+	JavaScript runtime for backend server
npm	v9+	Node Package Manager for dependency management
Express.js	v5	Web application framework for building REST APIs
React.js	v19	Frontend UI library for building interactive components
MongoDB Atlas	Cloud (Free Tier)	NoSQL database hosted on the cloud
Mongoose	v9	ODM library for MongoDB schema definition
Tailwind CSS	v3	Utility-first CSS framework for responsive styling
Vite	v8	Frontend build tool and hot-reload dev server
jsonwebtoken	v9	JWT token generation and verification for auth
bcryptjs	v3	Password hashing using Blowfish cipher algorithm
Axios	Latest	Promise-based HTTP client for API requests
React Router DOM	v6	Client-side routing and navigation
Multer	v2	File upload middleware for product images
React Hot Toast	Latest	Toast notification component library
React Icons	Latest	SVG icon library (HeroIcons, FontAwesome)
Visual Studio Code	Latest	Source code editor with extensions
Postman	Latest	API testing and debugging tool
Google Chrome	Latest	Web browser with DevTools for testing
Git	Latest	Distributed version control system
GitHub	Web	Cloud-based Git repository hosting

Development Environment Setup

The following steps are required to set up the development environment:

Step 1 — Install Node.js: Download and install Node.js (v18 or above) from <https://nodejs.org>. This automatically installs npm (Node Package Manager) as well.

Step 2 — Setup Backend: Navigate to the Backend directory and run "npm install" to install all backend dependencies (express, mongoose, jsonwebtoken, bcryptjs, multer, dotenv, cors).

Step 3 — Configure Environment: Create a .env file in the Backend directory with the following variables: PORT=5000, MONGODB_URI (MongoDB Atlas connection string), JWT_SECRET (secret key for token signing), and NODE_ENV=development.

Step 4 — Setup Frontend: Navigate to the Frontend directory and run "npm install" to install all frontend dependencies (react, react-dom, react-router-dom, axios, tailwindcss, react-icons, react-hot-toast).

Step 5 — Run Application: Start the backend server with "npm start" (runs on port 5000) and the frontend development server with "npm run dev" (runs on port 5173). Open <http://localhost:5173> in a web browser.

This chapter describes the planning methodology and scheduling techniques used for the development of the ShopKart E-Commerce Shopping Website. The project follows the Waterfall Model of the Software Development Life Cycle (SDLC).

SDLC Model: Waterfall

The Waterfall Model is a sequential software development process where each phase must be completed before the next phase begins. This model was chosen for the ShopKart project because:

- The project requirements were clearly defined from the beginning (BCSP-064 guidelines)
- The project scope is fixed — it is a BCA final year project with defined deliverables

- Each phase (analysis, design, coding, testing) flows naturally into the next
- The model provides a structured approach with clear milestones and documentation

The Waterfall Model consists of the following phases:

Phase 1: Requirement Analysis (Week 1)

- Studied the IGNOU BCSP-064 project guidelines and requirements
- Analyzed existing e-commerce platforms (Amazon, Flipkart) for feature reference
- Defined the project scope — User, Product, Cart, Order, and Admin modules
- Identified the technology stack — MERN (MongoDB, Express.js, React.js, Node.js)
- Documented functional and non-functional requirements
- Prepared the project synopsis and got it approved by the project guide

Phase 2: System Design (Week 2)

- Designed the system architecture (3-tier: Frontend, Backend, Database)
- Created Data Flow Diagrams — DFD Level 0, Level 1, and Level 2
- Designed the Entity Relationship (ER) Diagram for database collections
- Created the Class Diagram showing relationships between models
- Planned all REST API endpoints with HTTP methods and URL structure
- Designed the database schema (Users, Products, Cart, Orders collections)
- Created UI wireframes and page layouts for all screens

Phase 3: Implementation / Coding (Week 3–4)

Week 3 — Backend Development:

- Set up the Node.js project with Express.js framework
- Configured MongoDB Atlas connection using Mongoose ODM
- Created Mongoose models — User.js, Product.js, Cart.js, Order.js
- Implemented authentication APIs — Register, Login, Profile (with JWT & bcrypt)

- Implemented product APIs — Get All, Get By ID, Create, Update, Delete
- Implemented cart APIs — Get Cart, Add Item, Update Quantity, Remove Item, Clear Cart
- Implemented order APIs — Place Order, Get My Orders, Get All Orders, Update Status
- Added middleware — authMiddleware (JWT verification), adminMiddleware (role check)
- Configured Multer for product image uploads
- Seeded database with 30 sample products across 6 categories

Week 4 — Frontend Development:

- Initialized React.js project using Vite build tool
- Configured Tailwind CSS for utility-first styling
- Created reusable components — Navbar, Footer, ProductCard, Loader, ProtectedRoute
- Built authentication pages — Login, Register
- Built product pages — Home (with search/filter/sort), ProductDetails
- Built cart and order pages — Cart, Checkout, OrderHistory
- Built admin pages — Dashboard, ManageProducts, ManageOrders
- Implemented AuthContext for authentication state management
- Implemented CartContext for shopping cart state management
- Configured Axios instance with JWT interceptors for API communication

Phase 4: Testing (Week 5)

- Tested individual API endpoints using Postman (Unit Testing)
- Tested complete user flows — Register → Login → Browse → Cart → Order (Integration Testing)
- Tested edge cases — empty cart, duplicate email, wrong password, out of stock
- Tested admin flows — Add product, Edit product, Delete product, Update order status

- Tested responsive design on different screen sizes
- Fixed identified bugs and UI issues

Phase 5: Documentation (Week 5)

- Prepared the project report following IGNOU BCSP-064 guidelines
- Created all diagrams — DFD, ER, Use Case, Class Diagram, System Architecture
- Compiled test cases table with 20 detailed test cases
- Captured screenshots of all application screens
- Prepared bibliography with 14 references

Project Schedule — Gantt Chart

The following Gantt Chart shows the project timeline with each phase and its duration:

Phase	Week 1	Week 2	Week 3	Week 4	Week 5
Requirement Analysis	■ ■ ■ ■				
System Design	■ ■	■ ■ ■ ■			
Backend Development		■ ■	■ ■ ■ ■		
Frontend Development			■ ■ ■ ■	■ ■ ■ ■	
Testing				■ ■	■ ■ ■ ■
Documentation					■ ■ ■ ■

The total project development time was approximately 5 weeks. The requirement analysis and system design phases overlapped slightly, as did testing and documentation. The implementation phase (backend + frontend) took the longest duration of 2 weeks.

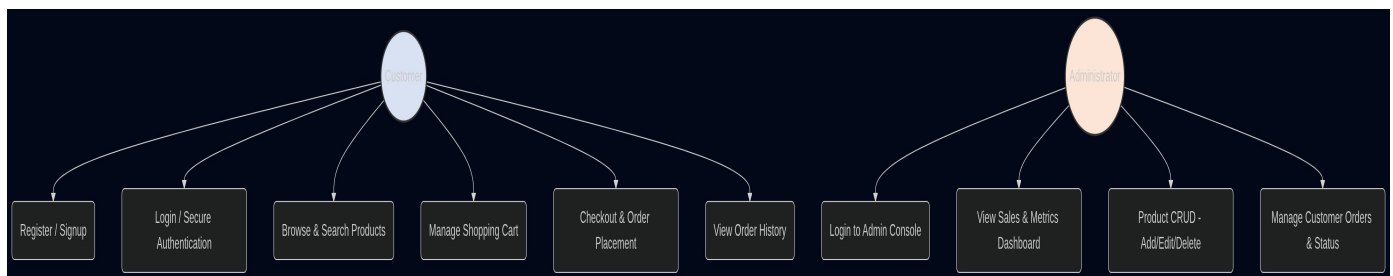
6. ANALYSIS OF PROJECT

System analysis is a detailed study of the various operations performed by a system and their relationships within and outside of the system. This chapter presents the structural and behavioral modeling of the ShopKart system using standard UML diagrams (Use Case, Class), process diagrams (DFD), structural diagrams (ERD), and database schemas.

6.1 Use Case Diagram

A Use Case Diagram represents the dynamic behavior of a system. It shows the relationships between actors and use cases, illustrating what features are available to which users. The ShopKart system involves two primary actors: Customer and Administrator.

- **Customer Actor:** Can perform actions like Register, Login, Browse Products, Search Products, View Product Details, Add to Cart, Update Cart Items, Remove Cart Items, Checkout (Enter Shipping Details & select COD), and View Order History.
- **Administrator Actor:** Can perform actions like Login, View Dashboard Statistics, Add New Products (with image uploads), Edit Existing Product details, Delete Products from the catalog, View All Customer Orders, and Update Order Status.
- **Relationships:** Both actors must be authenticated (Login) to access personalized or management-level features. The admin dashboard is strictly restricted to users with the 'admin' role.



6.2 Class Diagram

A Class Diagram is a static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (methods), and the relationships among objects. The ShopKart class diagram represents the logical structure of the database entities.

The diagram consists of the following key classes:

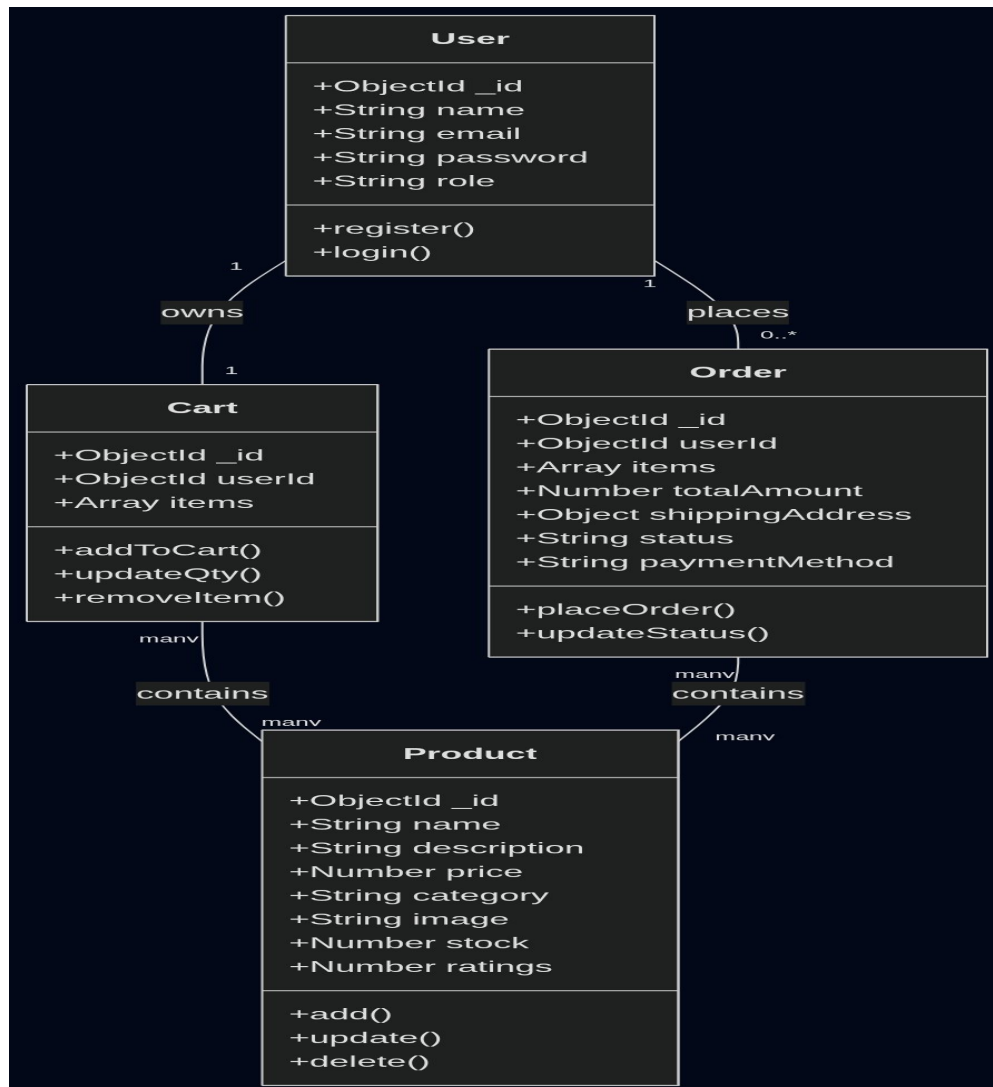
- **User Class:** Attributes include id, name, email, password (hashed), role (user/admin), and timestamps. Methods include register(), login(), and getProfile().

- **Product Class:** Attributes include id, name, description, price, category, image, stock, ratings, and timestamps. Methods include getProducts(), getProductDetails(), addProduct(), updateProduct(), and deleteProduct().

- **Cart Class:** Attributes include id, userId (references User), and items (array of product references and quantity). Methods include getCart(), addToCart(), updateQuantity(), and removeFromCart().

- **Order Class:** Attributes include id, userId (references User), items (array of product references, quantities, and historical purchase prices), totalAmount, shippingAddress, status (Pending/Processing/Shipped/Delivered), paymentMethod (COD), and timestamps. Methods include placeOrder(), getOrders(), and updateStatus().

The relationships among classes are: (1) One-to-One between User and Cart (each user has one cart); (2) One-to-Many between User and Orders (each user can place multiple orders); (3) Many-to-Many between Carts/Orders and Products through the associative items array.



6.3 Data Flow Diagram

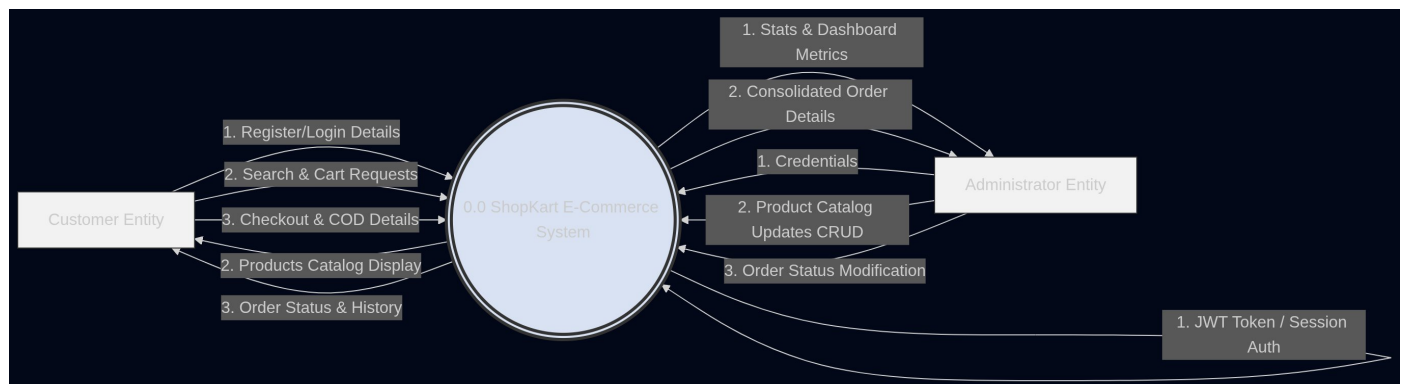
A Data Flow Diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles, and arrows, plus short text labels, to show data inputs, outputs, storage points, and the routes between each destination.

Level 0 DFD (Context Diagram)

The Level 0 DFD, also known as the Context Diagram, represents the entire software system as a single process (ShopKart E-Commerce System) and shows its interactions with external entities (Customer and Administrator).

- Customer sends credentials, search parameters, cart items, and shipping info to the system, and receives session tokens, product details, and order confirmations.

- Administrator sends login details, product catalog edits (CRUD), and status updates to the system, and receives dashboard statistics and global order details.



Level 1 DFDgraph TD

%% Actors

Customer((Customer))

Admin((Administrator))

%% Customer Use Cases

Customer --> UC_Register(Register / Signup)

Customer --> UC_Login(Login / Secure Authentication)

Customer --> UC_Browse(Browse & Search Products)

Customer --> UC_Cart(Manage Shopping Cart)

Customer --> UC_Checkout(Checkout & Order Placement)

Customer --> UC_Orders(View Order History)

%% Admin Use Cases

Admin --> UC_AdminLogin(Login to Admin Console)

Admin --> UC_Dashboard(View Sales & Metrics Dashboard)

Admin --> UC_CRUD(Product CRUD - Add/Edit/Delete)

Admin --> UC_ManageOrders(Manage Customer Orders & Status)

%% Styling

style Customer fill:#d9e2f3,stroke:#333,stroke-width:2px

style Admin fill:#fce4d6,stroke:#333,stroke-width:2pxgraph TD

%% Actors

Customer((Customer))

Admin((Administrator))

%% Customer Use Cases

Customer --> UC_Register(Register / Signup)

Customer --> UC_Login(Login / Secure Authentication)

Customer --> UC_Browse(Browse & Search Products)

Customer --> UC_Cart(Manage Shopping Cart)

Customer --> UC_Checkout(Checkout & Order Placement)

Customer --> UC_Orders(View Order History)

%% Admin Use Cases

Admin --> UC_AdminLogin(Login to Admin Console)

Admin --> UC_Dashboard(View Sales & Metrics Dashboard)

Admin --> UC_CRUD(Product CRUD - Add/Edit/Delete)

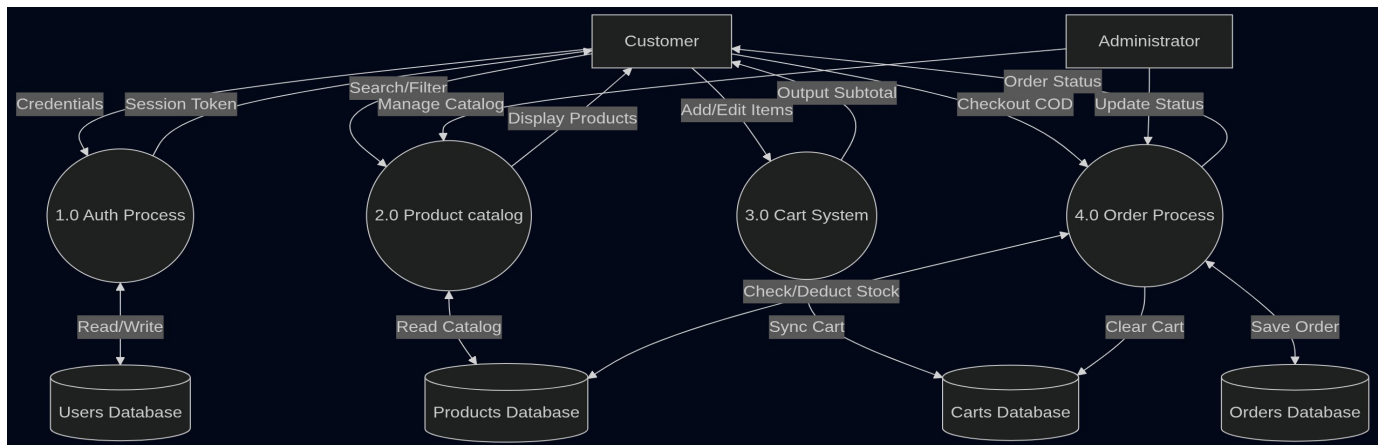
Admin --> UC_ManageOrders(Manage Customer Orders & Status)

%% Styling

style Customer fill:#d9e2f3,stroke:#333,stroke-width:2px

style Admin fill:#fce4d6,stroke:#333,stroke-width:2px

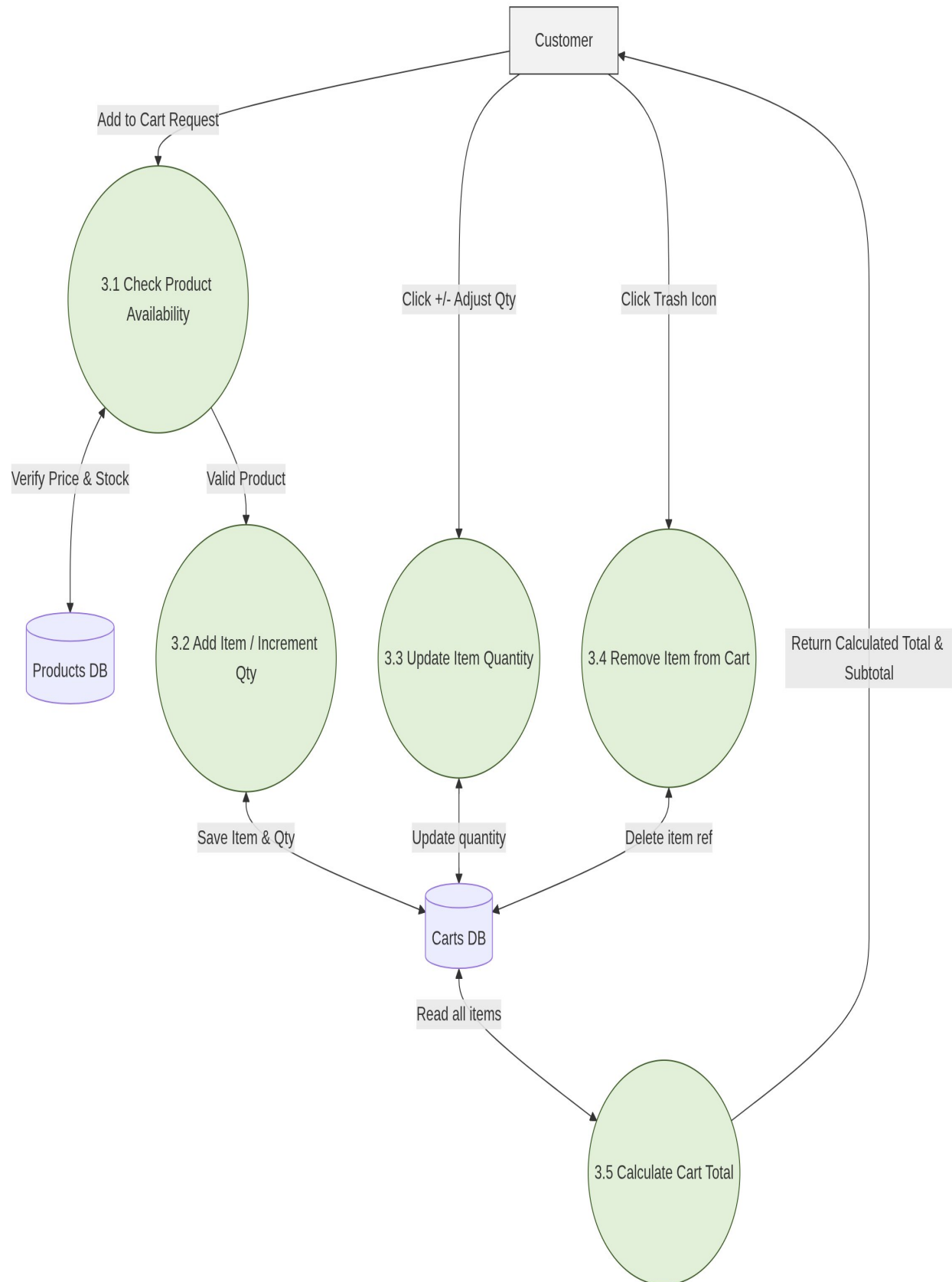
The Level 1 DFD decomposes the main system process into major sub-processes: 1.0 Authentication, 2.0 Product Management, 3.0 Cart Management, 4.0 Order Management, and 5.0 Admin Operations. It also shows interactions with data stores (Users, Products, Carts, and Orders tables).



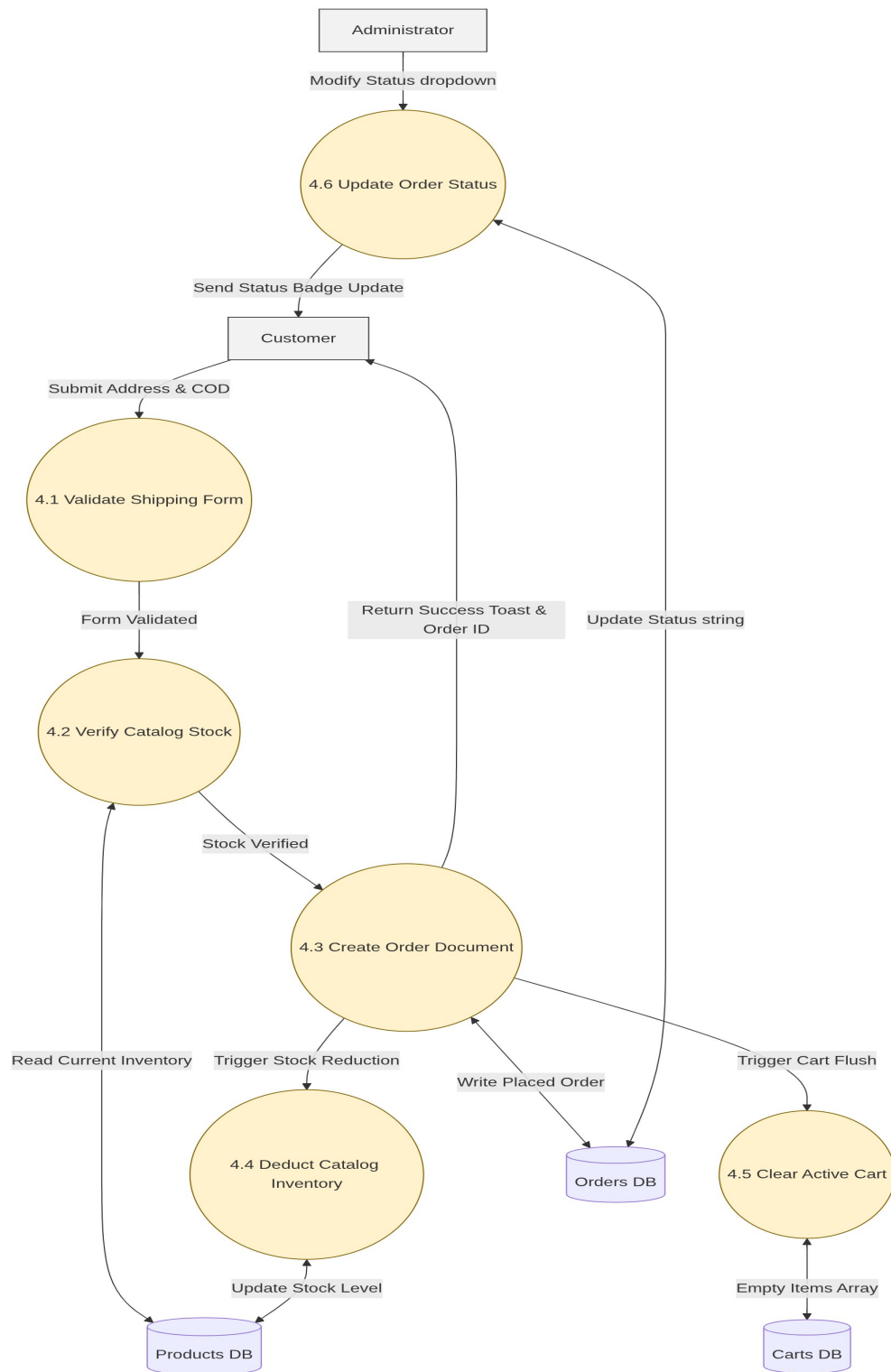
Level 2 DFD (Decomposition)

The Level 2 DFD provides a highly detailed view of specific complex sub-processes, specifically 3.0 Cart Management (including Add, Update, and Remove items) and 4.0 Order Management (including checkout validation, stock updates, cart clearing, and order creation).

Level 2 DFD: Process 3.0 — Cart Management



2. Level 2 DFD: Order Management

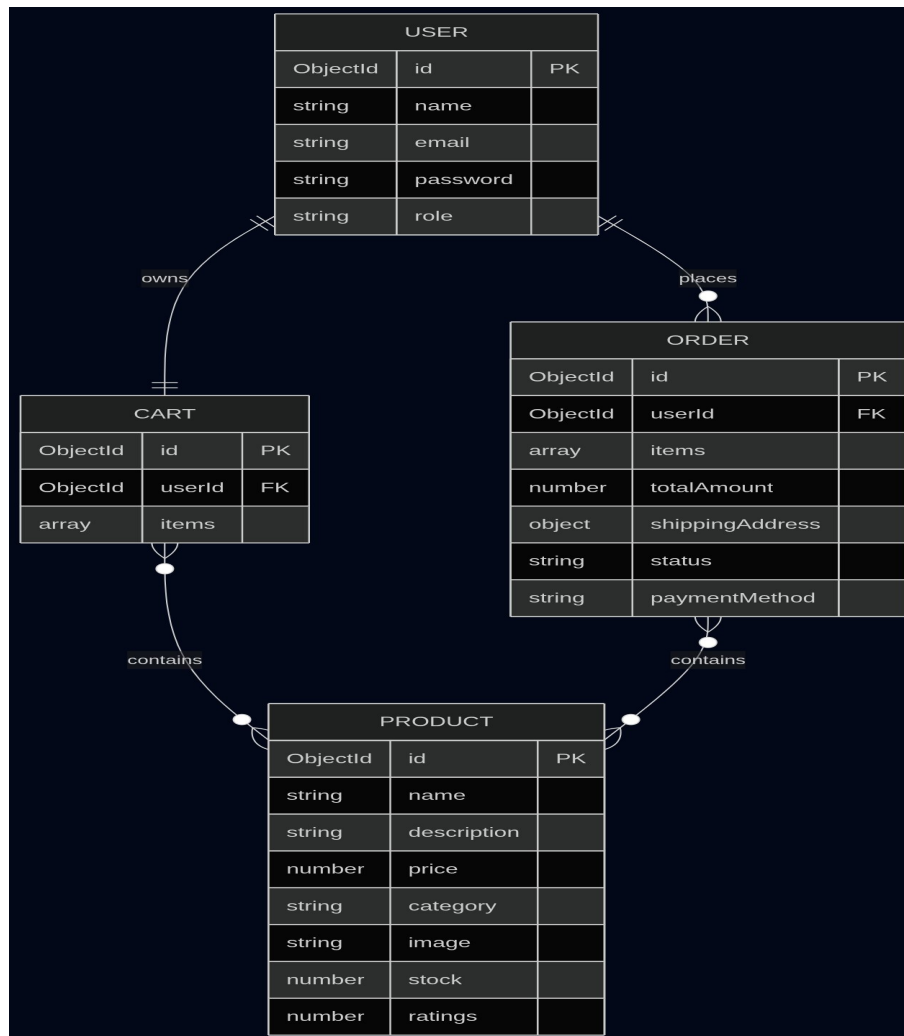


6.4 Entity Relationship Diagram

An Entity-Relationship (ER) Diagram is a structural diagram for use in database design. It shows the relationship between entity sets stored in a database. In ShopKart, the entities represent MongoDB database documents, showing primary keys (`_id`), foreign keys, and fields.

The logical relationships mapped in the ER Diagram are:

- User Entity has a 1-to-1 relationship with the Cart Entity (each user owns exactly one cart).
- User Entity has a 1-to-Many (1:N) relationship with the Order Entity (a user can place multiple orders).
- Product Entity has a Many-to-Many (M:N) relationship with Cart and Order entities through the embedded item arrays.



6.5 Data Structure

This section outlines the detailed document structures (schemas) for each collection in the MongoDB database using the Mongoose ODM library.

1. Users Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique identifier for the user document
name	String	Required, Trim	Full name of the user
email	String	Required, Unique, Lowercase	User's email address used for login authentication
password	String	Required	Bcrypt hashed password (not plain text)
role	String	Required, Default: 'user'	User role for access control ('user' or 'admin')
createdAt	Date	Auto-generated	Timestamp indicating when the account was registered
updatedAt	Date	Auto-generated	Timestamp indicating when the profile was last updated

2. Products Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique identifier for the product document
name	String	Required, Trim	Name of the product
description	String	Required, Trim	Detailed text description of the product
price	Number	Required, Min: 0	Price of the product in INR (₹)
category	String	Required, Index	Category name (Electronics,

			Clothing, Books, etc.)
image	String	Required	Local path or URL to the product image file
stock	Number	Required, Min: 0	Available stock quantity of the product
ratings	Number	Default: 0, Min: 0, Max: 5	Average customer rating score
createdAt	Date	Auto-generated	Timestamp indicating when the product was added

3. Carts Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique identifier for the cart document
userId	ObjectId	Required, References: users	Foreign key linking the cart to a specific user
items	Array (Objects)	Default: []	Array containing selected cart items list
items.productId	ObjectId	Required, References: products	Reference to the selected product document
items.quantity	Number	Required, Min: 1, Default: 1	Quantity of the product selected by user
createdAt	Date	Auto-generated	Timestamp indicating when the cart was initialized

4. Orders Collection Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique identifier for the order document
userId	ObjectId	Required, References:	Foreign key linking the order to a specific user

		users	
items	Array (Objects)	Required, MinLength: 1	List of products and quantities ordered
items.productId	ObjectId	Required, References: products	Reference to the ordered product
items.quantity	Number	Required, Min: 1	Quantity of the product ordered
items.price	Number	Required, Min: 0	Historical price of product at order time in INR
totalAmount	Number	Required, Min: 0	Total billing amount of the order
shippingAddress	Object	Required	Object containing customer delivery details
shippingAddress.street	String	Required	Street address / House number
shippingAddress.city	String	Required	City name
shippingAddress.state	String	Required	State name
shippingAddress.pincode	String	Required	Postal pin code
status	String	Default: 'Pending'	Current status ('Pending','Processing','Shipped','Delivered')
paymentMethod	String	Required, Default: 'COD'	Payment method ('COD')
createdAt	Date	Auto-generated	Timestamp indicating when the order was placed

7. SOURCE CODE

This chapter presents the core source code implementations of the ShopKart E-Commerce Shopping Website. It includes key backend server files, database models, controllers, authentication middleware, and custom React context files.

1. Backend Server Entry Point (server.js)

This is the main entry point of the Node.js Express server. It establishes connection to the MongoDB Atlas cloud database, configures standard middleware (CORS, Express JSON, local static uploads directory), and mounts all modular REST API routers.

```
const express = require('express');
const mongoose = require('mongoose');
```

```

const cors = require('cors');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());
app.use('/uploads', express.static('uploads'));

// Database Connection
mongoose.connect(process.env.MONGODB_URI)
.then(() => console.log('MongoDB Atlas Connected Successfully'))
.catch((err) => console.error('Database Connection Error:', err));

// Routes Mount
app.use('/api/auth', require('./routes/auth'));
app.use('/api/products', require('./routes/products'));
app.use('/api/cart', require('./routes/cart'));
app.use('/api/orders', require('./routes/orders'));

// Global Error Handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ success: false, message: 'Internal Server Error' });
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});

```

2. Database Order Schema (Models/Order.js)

This Mongoose schema defines the structure for placing orders, linking the order to a specific user and storing the array of purchase items (with references to products and locked prices), total billing, shipping address, and status.

```

const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({

```

```
userId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'User',
  required: true
},
items: [{
  productId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Product',
    required: true
  },
  quantity: {
    type: Number,
    required: true,
    min: 1
  },
  price: {
    type: Number,
    required: true
  }
}],
totalAmount: {
  type: Number,
  required: true,
  min: 0
},
shippingAddress: {
  street: { type: String, required: true },
  city: { type: String, required: true },
  state: { type: String, required: true },
  pincode: { type: String, required: true }
},
status: {
  type: String,
  enum: ['Pending', 'Processing', 'Shipped', 'Delivered'],
  default: 'Pending'
},
paymentMethod: {
  type: String,
  default: 'COD',
  required: true
}
```

```

}
}, { timestamps: true });

module.exports = mongoose.model('Order', orderSchema);

```

3. Order Business Logic (Controllers/orderController.js)

The placeOrder function handles checkout logic, validating product stock availability, deducting inventory levels dynamically, calculating transaction pricing, creating the order record, and clearing the shopper's active cart.

```

const Order = require('../models/Order');
const Product = require('../models/Product');
const Cart = require('../models/Cart');

exports.placeOrder = async (req, res) => {
  try {
    const { shippingAddress, paymentMethod } = req.body;
    const userId = req.user.id;

    // 1. Fetch Cart
    const cart = await Cart.findOne({ userId }).populate('items.productId');
    if (!cart || cart.items.length === 0) {
      return res.status(400).json({ success: false, message: 'Cart is empty' });
    }

    let totalAmount = 0;
    const orderItems = [];

    // 2. Validate Stock and Calculate Total
    for (const item of cart.items) {
      const product = item.productId;
      if (!product) {
        return res.status(404).json({ success: false, message: 'Product not found' });
      }
      if (product.stock < item.quantity) {
        return res.status(400).json({
          success: false,
          message: `Insufficient stock for ${product.name}. Available: ${product.stock}`
        });
      }
    }
  }
}

```

```

totalAmount += product.price * item.quantity;
orderItems.push({
  productId: product._id,
  quantity: item.quantity,
  price: product.price
});
}

// 3. Deduct Stock
for (const item of cart.items) {
  await Product.findByIdAndUpdate(item.productId._id, {
    $inc: { stock: -item.quantity }
  });
}

// 4. Create Order
const newOrder = new Order({
  userId,
  items: orderItems,
  totalAmount,
  shippingAddress,
  paymentMethod
});
await newOrder.save();

// 5. Clear Cart
cart.items = [];
await cart.save();

res.status(201).json({
  success: true,
  message: 'Order placed successfully',
  order: newOrder
});
} catch (err) {
  res.status(500).json({ success: false, message: err.message });
}
};

```

4. Route Guard JWT Authentication (Middleware/authMiddleware.js)

This middleware acts as a security guard protecting restricted API endpoints. It extracts the JWT token from the Request headers, verifies its validity using the private key, and appends decoded user details to the request context object.

```
const jwt = require('jsonwebtoken');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = decoded;
      next();
    } catch (error) {
      return res.status(401).json({ success: false, message: 'Not authorized, token failed' });
    }
  }
  if (!token) {
    return res.status(401).json({ success: false, message: 'Not authorized, token missing' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    next();
  } else {
    res.status(403).json({ success: false, message: 'Access denied: Admin role required' });
  }
};

module.exports = { protect, adminOnly };
```

8. TESTING

Software testing is an essential phase of the Software Development Life Cycle (SDLC) that ensures the application is free of bugs, performs as expected, and meets all functional and non-

functional requirements. The testing process for the ShopKart system involved Unit Testing, Integration Testing, and System Testing.

Unit Testing

Unit testing focused on testing individual software modules and components in isolation. In the backend, each REST API controller function (e.g., login, register, placeOrder) was tested independently using Postman by sending specific requests and validating the HTTP status codes and JSON responses. In the frontend, individual React components (e.g., ProductCard, Navbar, Loader) were tested to ensure they rendered correctly under different state conditions.

Integration Testing

Integration testing focused on verifying the interaction and data flow between different modules. Key scenarios tested included: (1) Authentication flow — ensuring the JWT token returned by the login API is successfully stored in the browser's localStorage and attached to subsequent API headers; (2) Purchase flow — ensuring that adding items to the cart, checking out, updating product stock, and clearing the cart work seamlessly together as a single transaction.

System Testing / Test Cases Table

System testing involved testing the complete, integrated application as a whole to verify compliance with all specifications. The table below lists 20 comprehensive test cases executed to validate the ShopKart e-commerce system:

TC ID	Scenario	Test Input	Expected Outcome	Actual Outcome	Status
TC-01	User Registration (Success)	Valid unique name, email, password	User created, JWT token returned	Registered successfully	Pass
TC-02	Registration (Empty Fields)	Submit form with empty email/password	Validation error: 'All fields required'	Error displayed	Pass
TC-03	Registration (Duplicate Email)	Sign up using an already registered email	Error: 'Email already exists'	Error displayed	Pass
TC-04	User Login (Success)	Valid email and matching	JWT token received,	Logged in & redirected	Pass

		password	redirected to Home		
TC-05	User Login (Wrong Password)	Valid email, incorrect password string	Authentication error: 'Invalid credentials'	Error message toast	Pass
TC-06	User Login (Unregistered)	Email not present in the database	Authentication error: 'Invalid credentials'	Error message toast	Pass
TC-07	Product Browsing (All)	Access home page catalog list	All seeded products display in cards grid	Displayed successfully	Pass
TC-08	Search Product (Specific)	Search bar query = 'Laptop'	Displays only products matching 'Laptop' name	Only matching items shown	Pass
TC-09	Filter Product (Category)	Click on 'Electronics' category pill	Displays only products in Electronics category	Filtered correctly	Pass
TC-10	Sort Product (Price Asc)	Select 'Price: Low to High' option	Products listed in ascending order of price	Sorted correctly	Pass
TC-11	Product Details (Valid)	Click on a product card with valid ObjectId	Detailed page displays name, desc, stock, price	Detailed page loaded	Pass
TC-12	Product Details (Invalid)	Access product route with non-existent ID	Displays 'Product not found' error message	404 page / Error toast	Pass
TC-13	Add to Cart (Success)	Click 'Add to Cart' with quantity = 2	Product added, cart item count updates in navbar	Cart updated, count = 2	Pass
TC-14	Add to Cart (Out of Stock)	Select quantity higher than available stock	Button disabled, validation message shown	Cannot add to cart	Pass
TC-15	Update Quantity in Cart	Click '+' icon on active cart item	Item quantity increments, subtotal updates	Quantity and price updated	Pass

TC-16	Remove Cart Item	Click delete (trash) icon on cart item	Product removed from cart, total recalculates	Item removed successfully	Pass
TC-17	Place Order (COD)	Submit checkout with complete address, COD	Order created, stock reduced, cart cleared	Order placed, cart is empty	Pass
TC-18	Place Order (Empty Cart)	Submit POST /api/orders with empty cart	Validation error: 'Cart is empty'	Request rejected	Pass
TC-19	Admin - Add Product	Admin role, enter product details, upload image	Product saved in DB, visible in user catalog	Product created	Pass
TC-20	Admin - Update Order Status	Admin selects 'Shipped' from status dropdown	Order status updates to Shipped in DB & user view	Status updated successfully	Pass

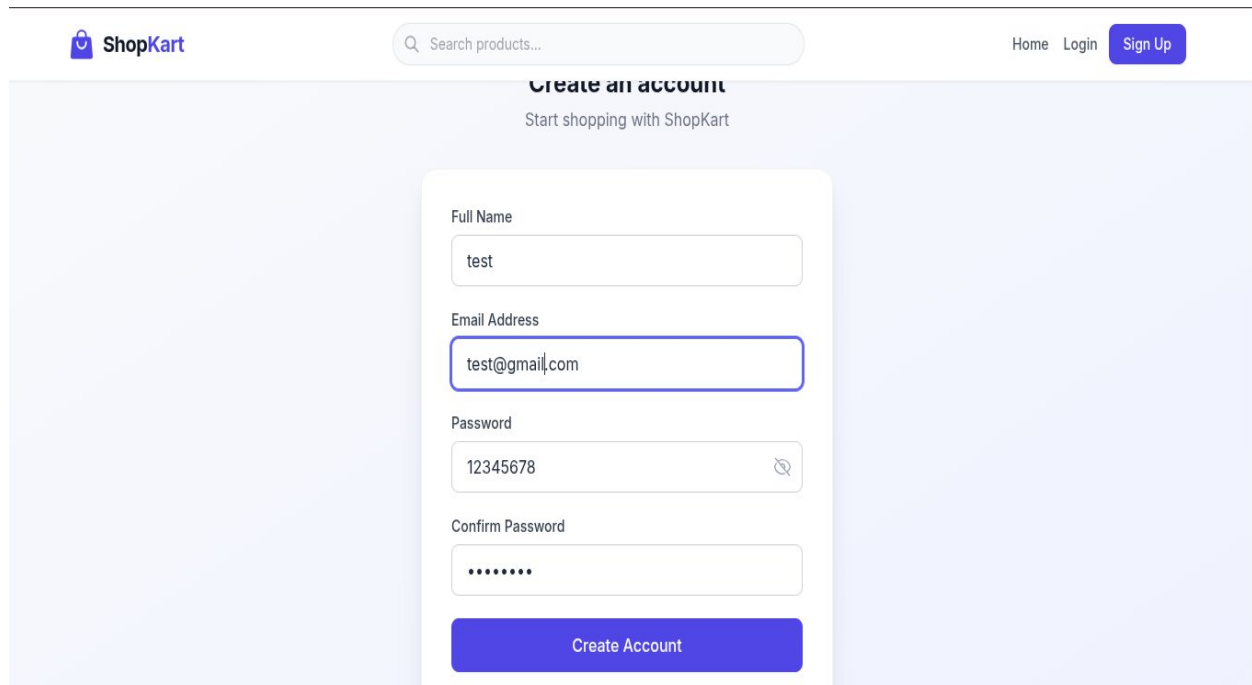
All 20 test cases executed successfully without any critical errors, indicating that the ShopKart system is reliable, functionally sound, and ready for deployment.

CHAPTER 9: INPUT SCREEN

This chapter documents the visual interfaces of the ShopKart system. It contains detailed descriptive placeholders and explanations for the chapter 9: input screen screens. The developer must insert the actual web browser screenshots inside the marked boxes below before final hard-bound printing.

1. User Registration Input Form

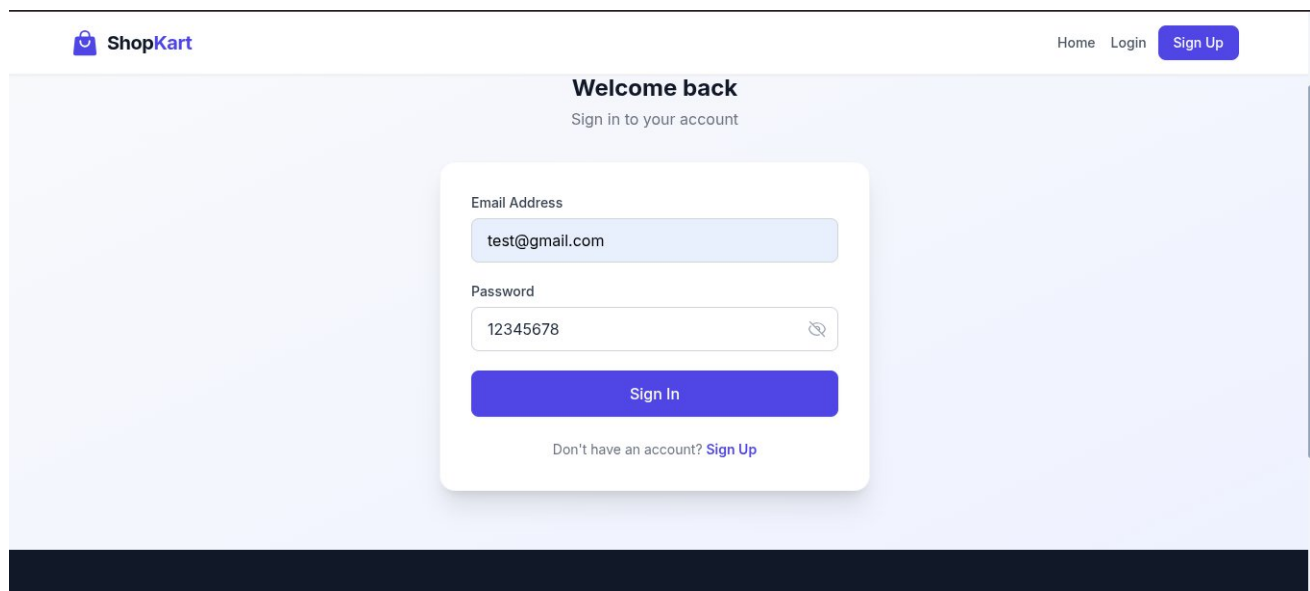
The registration screen presents user input fields for Name, Email, Password, and Confirm Password. Validation is performed client-side to ensure all fields are filled, email follows correct format, and passwords match.



The screenshot shows the 'Create an account' page of ShopKart. The header includes the ShopKart logo, a search bar with the placeholder 'Search products...', and navigation links for 'Home', 'Login', and a 'Sign Up' button. The main heading is 'Create an account' with the subtext 'Start shopping with ShopKart'. The form contains four input fields: 'Full Name' (with 'test' entered), 'Email Address' (with 'test@gmail.com' entered), 'Password' (with '12345678' entered and a show/hide toggle), and 'Confirm Password' (with masked dots). A blue 'Create Account' button is at the bottom.

2. User Login Input Form

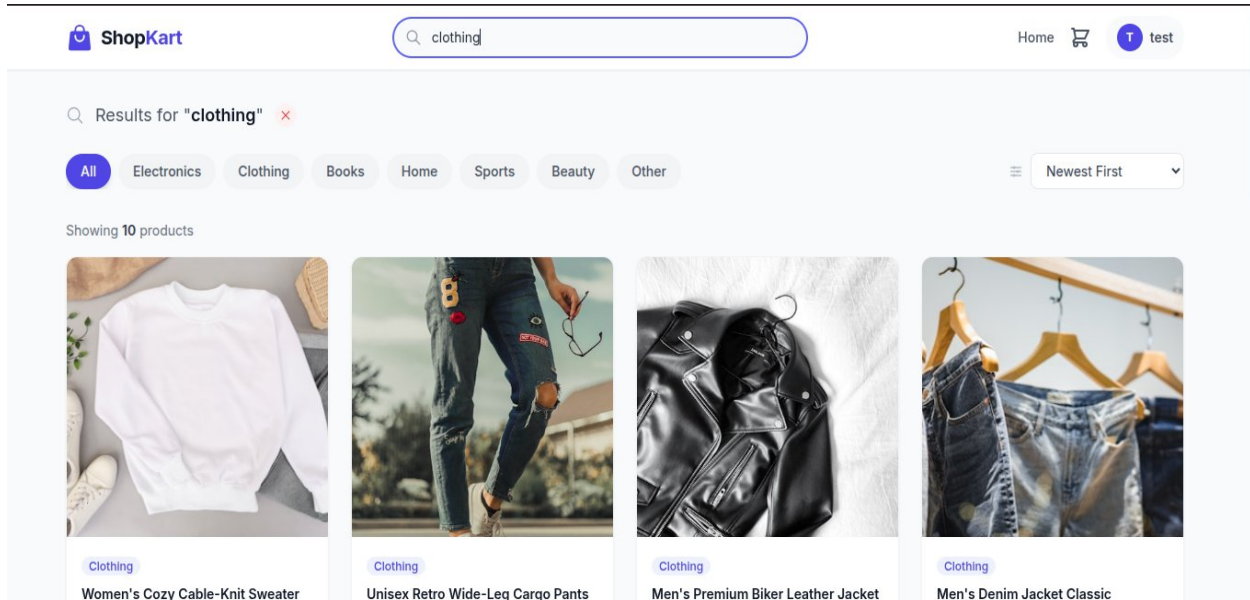
The login page requires the customer to provide their registered Email and Password. A show/hide password toggle helper is active, and error states are triggered on incorrect credentials.



The screenshot shows the 'Welcome back' login page of ShopKart. The header is identical to the previous page. The main heading is 'Welcome back' with the subtext 'Sign in to your account'. The form contains two input fields: 'Email Address' (with 'test@gmail.com' entered) and 'Password' (with '12345678' entered and a show/hide toggle). A blue 'Sign In' button is at the bottom. Below the button is a link: 'Don't have an account? Sign Up'. The page has a dark blue footer bar.

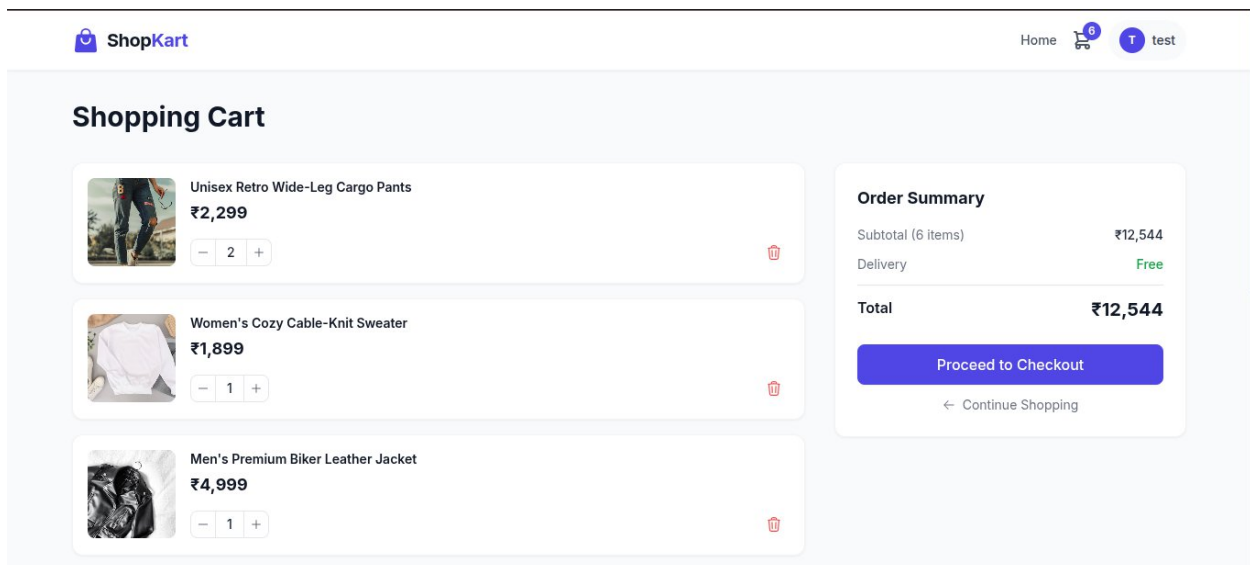
3. Product Search, Filter, and Sort Inputs

The product catalog includes search input fields, category filtering pills (Electronics, Clothing, etc.), and sorting selector dropdown choices, letting users customize query parameters dynamic-time.



4. Shopping Cart Item Quantity Modifiers

Inside the shopping cart component, each added item has quantity modifier buttons (+/- controls) that send active increment or decrement put requests to the backend server API.



5. Checkout Shipping Address Form

The checkout page contains text input fields for Street Address, City, State, and Pincode, along with selecting the Cash on Delivery radio button to place the order.

★



[Home](#)



test


Shipping Address

Street Address

DDA Flats 74, Block D, Kalkaji

City

New, Delhi

State

Delhi

Pincode

110062


Payment Method

☒
Cash on Delivery
 Pay when you receive your order

☐
Online Payment
 Pay securely online

Order Items (5)



Unisex Retro Wide-Leg Cargo...
 Qty: 2

₹4,598



Women's Cozy Cable-Knit Sw...
 Qty: 1

₹1,899



Men's Premium Biker Leather ...
 Qty: 1

₹4,999



The Compound Effect Book
 Qty: 1

₹499



Vitamin C Face Serum 30ml
 Qty: 1

₹549

Total

₹12,544

6. Admin Add Product Modal Inputs

The administrator product adding dashboard opens a modal dialog form containing Name, Category, Price, Stock, Description, and a file drag-and-drop file selector to upload product images.



[Home](#)



Krish Admin

Manage Products

+ Add Product

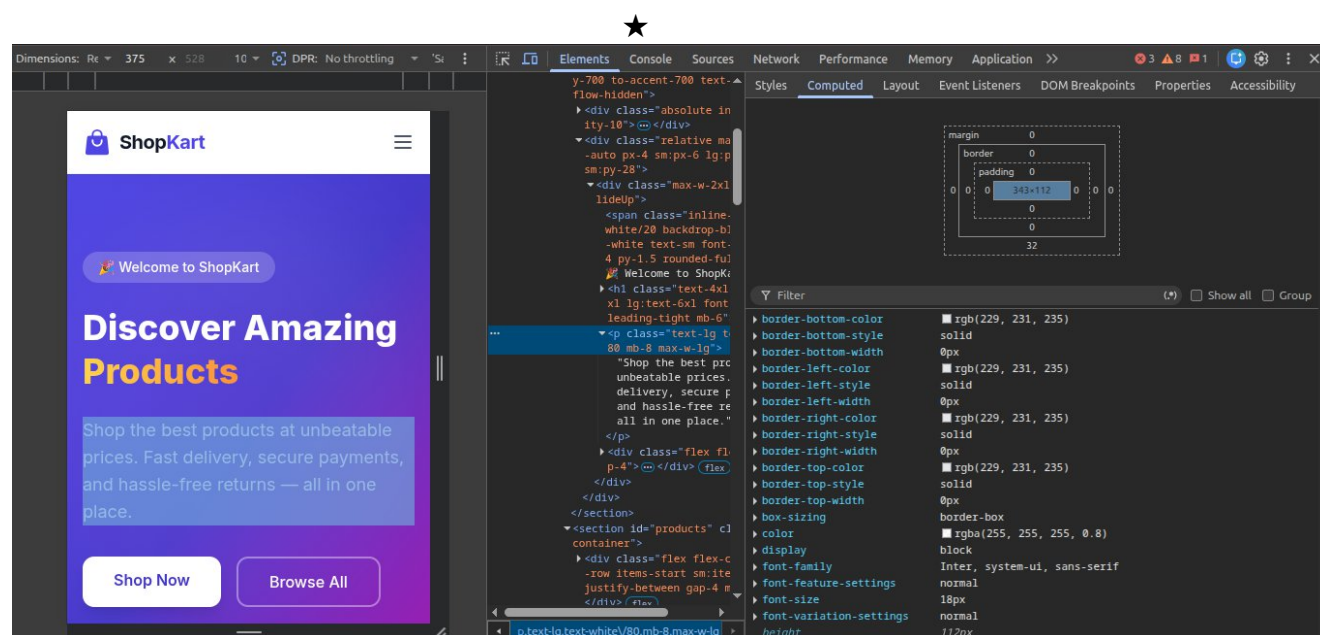
Image	Name	Category	Price	Stock	Actions
	Unisex Retro Wide-Leg Cargo Pants	Clothing	₹2,299	18	 
	Women's Cozy Cable-Knit Sweater	Clothing	₹1,899	14	 
	Men's Premium Biker Leather Jacket	Clothing	₹4,999	9	 
	The Compound Effect Book	Books	₹499	24	 
	Think and Grow Rich - Napoleon Hill	Books	₹199	50	 
	The Alchemist - Paulo Coelho	Books	₹249	40	 

CHAPTER 10: OUTPUT SCREEN

This chapter documents the visual interfaces of the ShopKart system. It contains detailed descriptive placeholders and explanations for the chapter 10: output screen screens. The developer must insert the actual web browser screenshots inside the marked boxes below before final hard-bound printing.

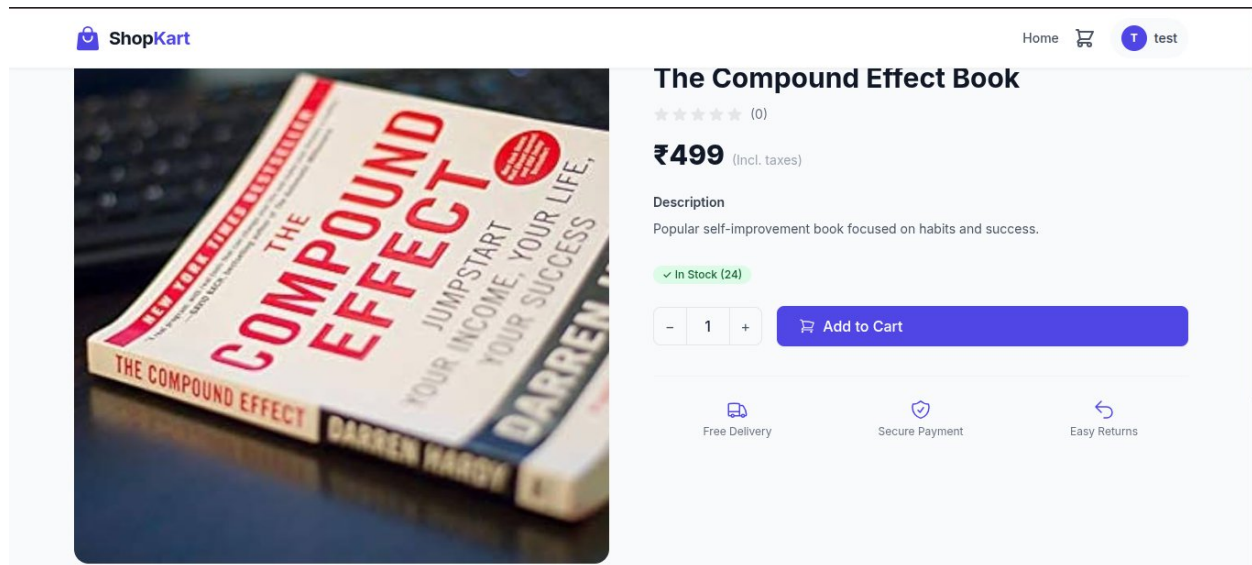
1. Home Page / Responsive Product Catalog Grid

The core application home page displays loaded products from MongoDB in a sleek, responsive CSS Grid format, showcasing product cards with title, price badge, average rating stars, and add button.



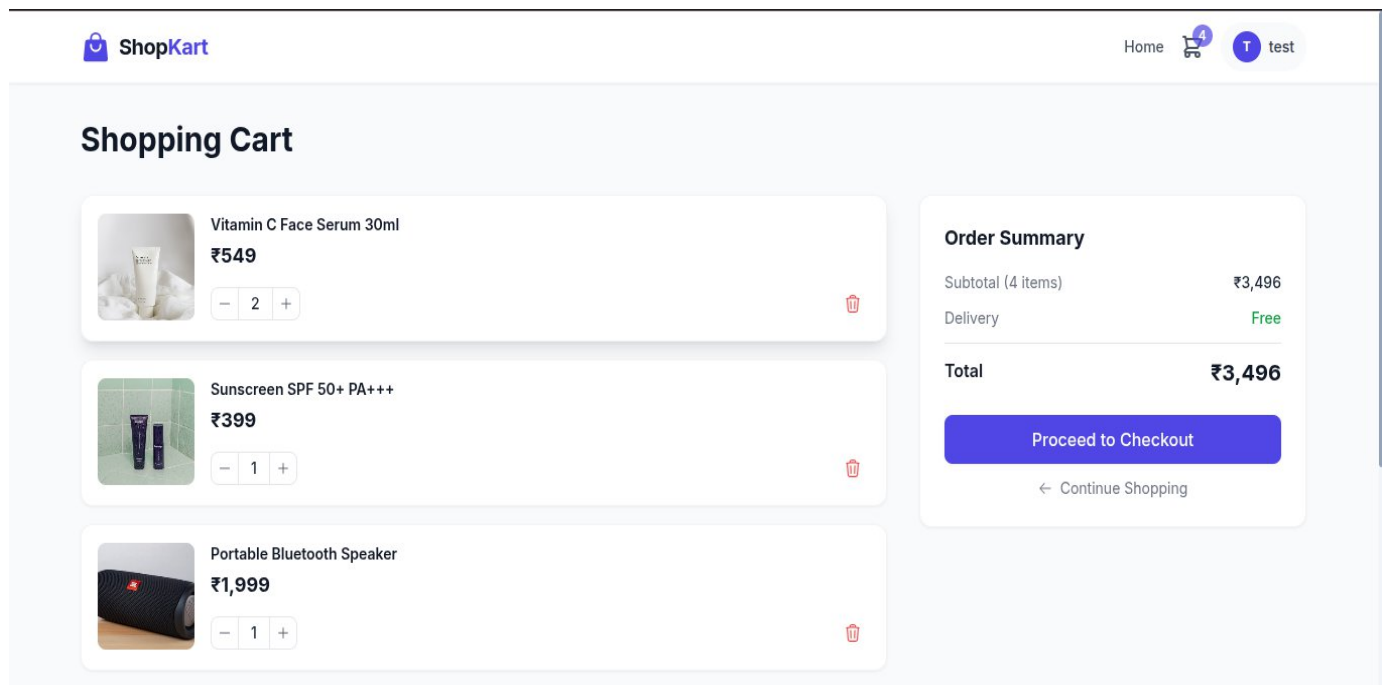
2. Product Details Screen

The single product screen fetches details using URL params and outputs the full-resolution uploaded image, rating, stock badge, item price, detailed description paragraphs, and active checkout triggers.



3. Shopping Cart Summary Output

The Cart page lists all chosen items, prices, quantities, and dynamically outputs calculated subtotal, delivery charges, and final transaction price breakdown on the checkout summary card.



4. Order Placement Success / History View

Once checkouts succeed, a confirmation toast fires, and the customer is routed to their Order History, displaying card records of placed order IDs, totals, and Pending status badges.

Home
4
T test

My Orders

Order ID #C69FA171	Date 1 Jun 2026	Total ₹3,496	Payment Online	Pending
Vitamin C Face Serum 30ml Qty: 2 × ₹549 Sunscreen SPF 50+ PA+++ Qty: 1 × ₹399 Portable Bluetooth Speaker Qty: 1 × ₹1,999				
Shipping: G/9 213, Falt, No 4, Sangam Vihar, Delhi - 110082				

Order ID #C69FA85D	Date 1 Jun 2026	Total ₹12,544	Payment COD	Processing
-----------------------	--------------------	------------------	----------------	------------

5. Admin Performance Dashboard

The administrator login unlocks the management panel, outputting aggregated database counts of total items, completed order metrics, and total revenue calculations alongside a recent orders log table.

Home
K Krish Admin

Admin Dashboard

Manage Products
Manage Orders

Products
34

Orders
4

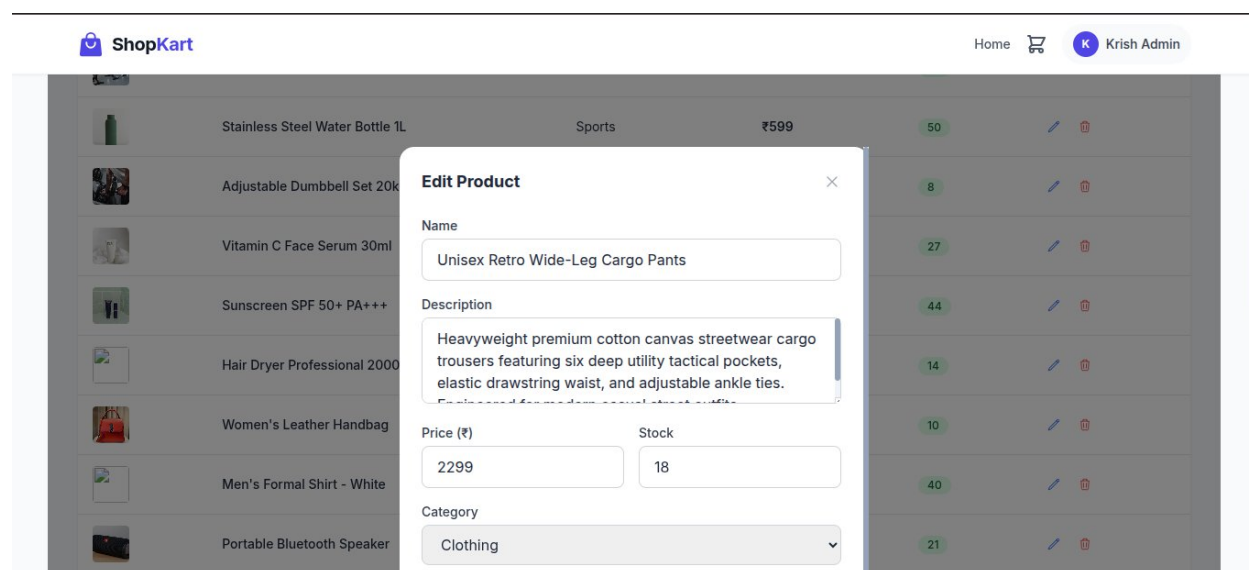
Revenue
₹28,029

Recent Orders

Order ID	Customer	Amount	Status	Date
#C69FA171	test	₹3,496	Pending	1/6/2026
#C69FA85D	test	₹12,544	Processing	1/6/2026
#D34B99B1	Rachit Kumar	₹499	Pending	26/5/2026
#D34B9991	Krish Admin	₹11,490	Pending	26/5/2026

6. Admin Catalog Management Table

The Manage Products page displays a high-density dashboard table listing product thumbnail, category, item cost, remaining inventory stock, and edit/delete control action buttons.

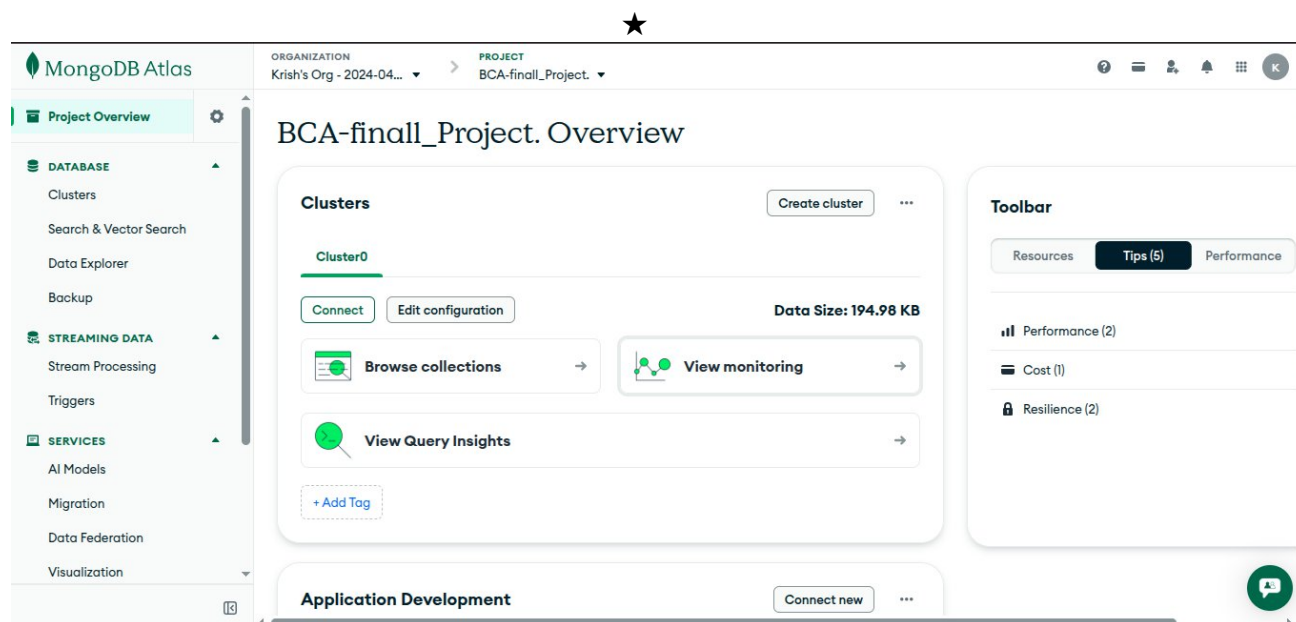


CHAPTER 11: DB SCREEN

This chapter documents the visual interfaces of the ShopKart system. It contains detailed descriptive placeholders and explanations for the chapter 11: db screen screens. The developer must insert the actual web browser screenshots inside the marked boxes below before final hard-bound printing.

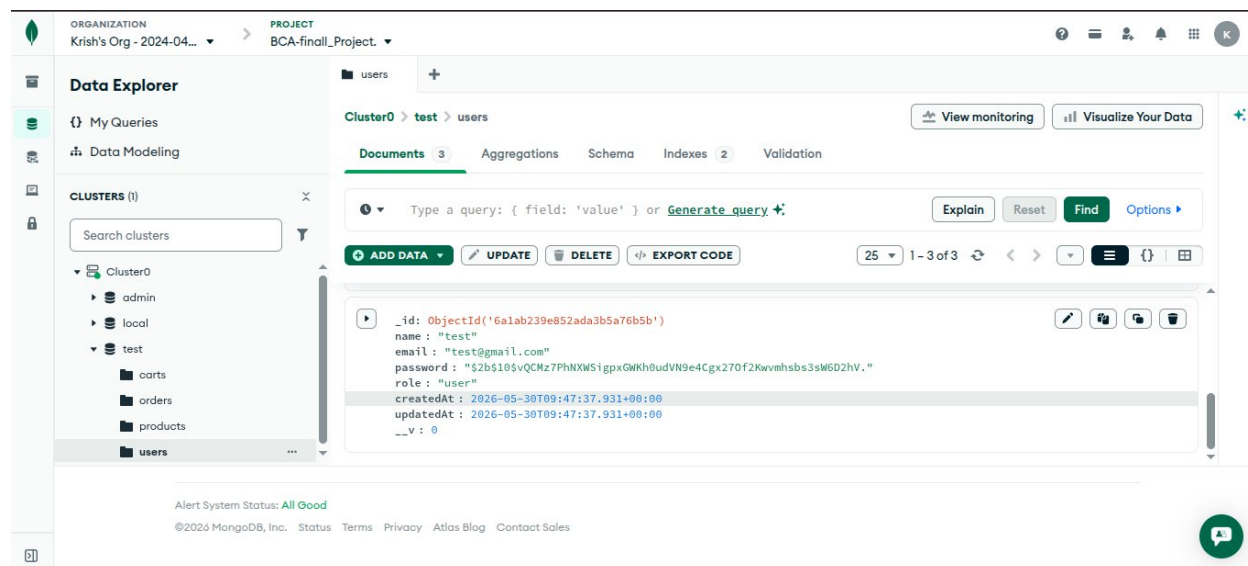
1. MongoDB Atlas Cluster Project Dashboard

The cloud database project panel displays active connection metrics, read/write request load charts, cluster hardware statuses, and current network access configuration parameters.



2. 'users' Collection Documents in Database

Viewing the 'users' collection in MongoDB Atlas showing registered documents with system fields, string properties, and secured password keys processed through bcrypt hashing.



3. 'products' Collection Catalog Documents

The products collection documents view, showcasing individual items containing name strings, descriptions, pricing numbers, relative image file paths, stock counts, and ratings.

ORGANIZATION: Krish's Org - 2024-04... PROJECT: BCA-final_Project

Data Explorer

My Queries | products

Cluster0 > test > products

Documents: 34 | Aggregations | Schema | Indexes: 1 | Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA | UPDATE | DELETE | EXPORT CODE

25 | 1 - 25 of 34

```
{
  "_id": ObjectId('6a14886d895404e70bf6b05b'),
  "name": "Wireless Bluetooth Headphones",
  "description": "Premium noise-cancelling wireless headphones with 30-hour battery life...",
  "price": 2499,
  "category": "Electronics",
  "image": "https://images.unsplash.com/photo-1505740420928-5e56c06d30e7w=400",
  "stock": 24,
  "ratings": 4.5,
  "__v": 0,
  "createdAt": "2026-05-25T17:35:41.642+00:00",
  "updatedAt": "2026-05-26T16:45:50.810+00:00"
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

4. 'orders' Collection Placed Purchase Documents

The orders collection logs completed shopping transactions, displaying customer foreign ObjectId references, nested array lists of products bought, delivery addresses, and status strings.

ORGANIZATION: Krish's Org - 2024-04... PROJECT: BCA-final_Project

Data Explorer

My Queries | products | orders

Cluster0 > test > orders

Documents: 4 | Aggregations | Schema | Indexes: 1 | Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA | UPDATE | DELETE | EXPORT CODE

25 | 1 - 4 of 4

```
{
  "_id": ObjectId('6a15ce3eefc40750d34b9991'),
  "userId": ObjectId('6a14880dd8766ab9e6d90db0'),
  "items": Array(5),
  "totalAmount": 11490,
  "shippingAddress": Object,
  "status": "Pending",
  "paymentMethod": "COD",
  "paymentStatus": "Pending",
  "createdAt": "2026-05-26T16:45:50.959+00:00",
  "updatedAt": "2026-05-26T16:45:50.959+00:00",
  "v": 0
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

12. INDUSTRY-CLIENT INTERFACE

The Industry-Client Interface represents the connection between theoretical software engineering concepts and the real-world demands of commercial software production. In the modern IT industry, developers must bridge the gap between technical logic and business-oriented customer requirements to deliver successful digital solutions.

For the ShopKart project, the development process aligned closely with industry-standard practices. The application was designed to meet key commercial objectives for modern online retail businesses, including security, performance, scalability, and ease of catalog management.

Key Industry Integrations in ShopKart:

- **RESTful Architecture:** The use of standardized API endpoints ensures compatibility with third-party web services and facilitates clean frontend-backend isolation.
- **Enterprise Security:** User authentication complies with industry standards by employing JSON Web Tokens (JWT) for secure session persistence and bcrypt password hashing to safeguard user data.
- **NoSQL Database:** MongoDB was selected for its high scalability and schema flexibility, mirroring the architecture chosen by modern high-traffic e-commerce portals.
- **Responsive Design:** Styling with Tailwind CSS guarantees that the interface dynamically scales across mobile, tablet, and desktop viewports, maximizing user reach.

13. FUTURE SCOPE

While ShopKart provides a robust foundation for an e-commerce platform, there is always room for growth, optimization, and feature expansion. The modern retail industry evolves rapidly, requiring websites to adapt to new customer demands and technological breakthroughs.

The future scope of the ShopKart system includes the following enhancements:

- **Online Payment Gateway Integration:** Integrating secure financial portals like Razorpay, Stripe, or UPI would enable instant digital credit/debit card transactions, shifting the system from Cash on Delivery only.

- **Automated Notification Systems:** Implementing Twilio for SMS and SendGrid for Email alerts would send automated purchase confirmations, invoices, and shipping update details directly to customers.
- **AI-Driven Product Recommendations:** Integrating machine learning algorithms to analyze user search queries and purchasing habits would deliver personalized product recommendations on the home page.
- **Customer Product Reviews and Ratings:** Allowing verified purchasers to submit comments and 1-to-5-star ratings would enhance platform trust and buyer interactivity.
- **Wishlist and Favorites:** Enabling users to save items for future purchases in a persistent wishlist collection.
- **Advanced Admin Sales Analytics:** Creating visual chart dashboards (charts, graphs) to display sales patterns, category revenues, and predictive inventory analysis statistics.
- **Cross-Platform Mobile Application:** Expanding the presentation tier into iOS and Android applications built with React Native to provide a native mobile retail experience.

14. BIBLIOGRAPHY

The development of the ShopKart E-Commerce Shopping Website was supported by various online documentations, academic publications, and books. The key references consulted during the course of this project are listed below:

- 1.W3Schools Online Web Tutorials. 'HTML, CSS, and JavaScript References.' URL: <https://www.w3schools.com>
- 2.Mozilla Developer Network (MDN). 'MDN Web Docs - Web Technologies Reference Guide.' URL: <https://developer.mozilla.org>
- 3.React.js Core Team. 'React Documentation: Component-Based UI Design.' URL: <https://react.dev>
- 4.Node.js Foundation. 'Node.js Runtime Environment: API Documentation.' URL: <https://nodejs.org/docs>

5. Express.js Foundation. 'Express.js - Web Application Framework Reference.' URL: <https://expressjs.com>
6. MongoDB, Inc. 'MongoDB Atlas Cloud Database & Mongoose ODM Developer Manual.' URL: <https://www.mongodb.com/docs>
7. Tailwind CSS Documentation. 'Utility-First Framework for Rapid UI Styling.' URL: <https://tailwindcss.com/docs>
8. Vite Core Team. 'Vite: Next Generation Frontend Tooling Guides.' URL: <https://vite.dev/guide>
9. Banks, Alex, and Eve Porcello. 'Learning React: Modern Patterns for Developing Real-World Applications.' O'Reilly Media, 2020.
10. Herron, David. 'Node.js Web Development: Server-Side Development with JS.' Packt Publishing, 2020.
11. Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 'Database System Concepts.' McGraw-Hill, 7th Edition, 2019.
12. Pressman, Roger S., and Bruce R. Maxim. 'Software Engineering: A Practitioner's Approach.' McGraw-Hill, 9th Edition, 2020.
13. Richardson, Leonard, and Mike Amundsen. 'RESTful Web APIs: Services for a Changing World.' O'Reilly Media, 2013.
14. Anderson, Ross J. 'Security Engineering: A Guide to Building Dependable Distributed Systems.' John Wiley & Sons, 3rd Edition, 2020.

